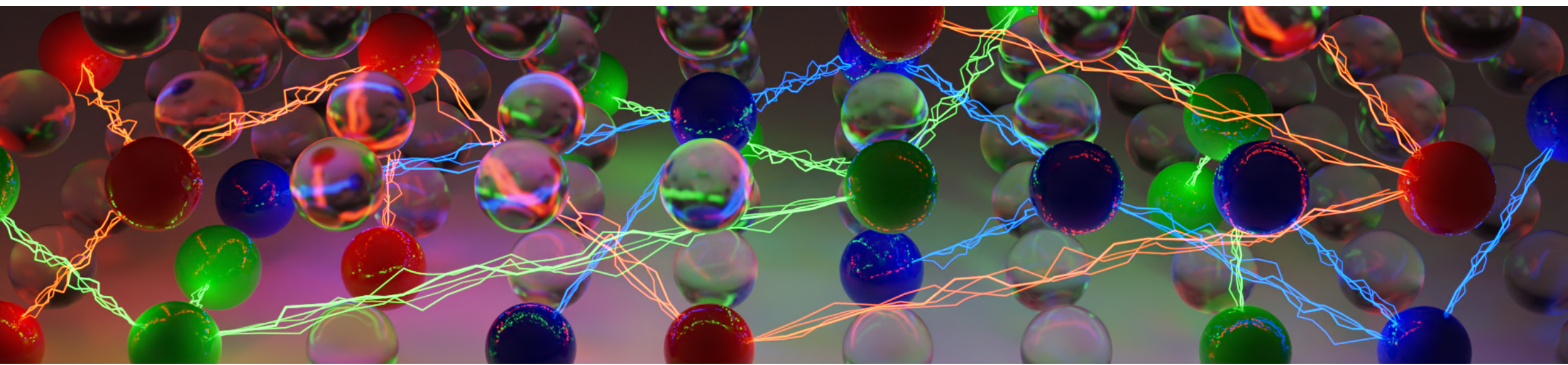
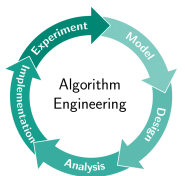




Practical SAT Solving

Lecture 8 – Parallel SAT Solving

Markus Iser, [Dominik Schreiber](#) | June 16, 2025



SAtRes
Scalable Automated Reasoning

Overview

Recap Lecture 7

- Propagation-based Redundancy notions
- Proof systems: Resolution, Extended Resolution, Blocked Clauses, Implication-based Redundancy
- Autarkies, Conditional Autarkies, and Satisfaction Driven Clause Learning (SDCL)

Today: Parallel SAT Solving

Parallel SAT solving approaches

- Basic search space splitting, Clause sharing, Cube&Conquer, Portfolio solvers

A deep dive into Mallob

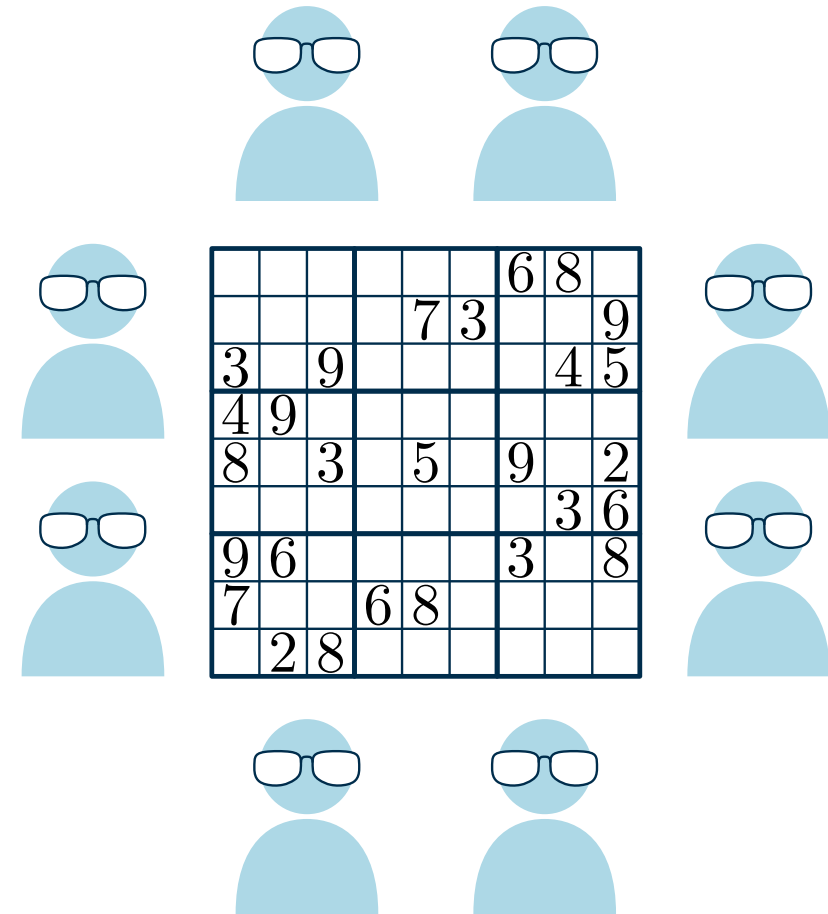
- Overview, Scalable clause sharing, Experiments and insights

Parallel Solving: An analogy

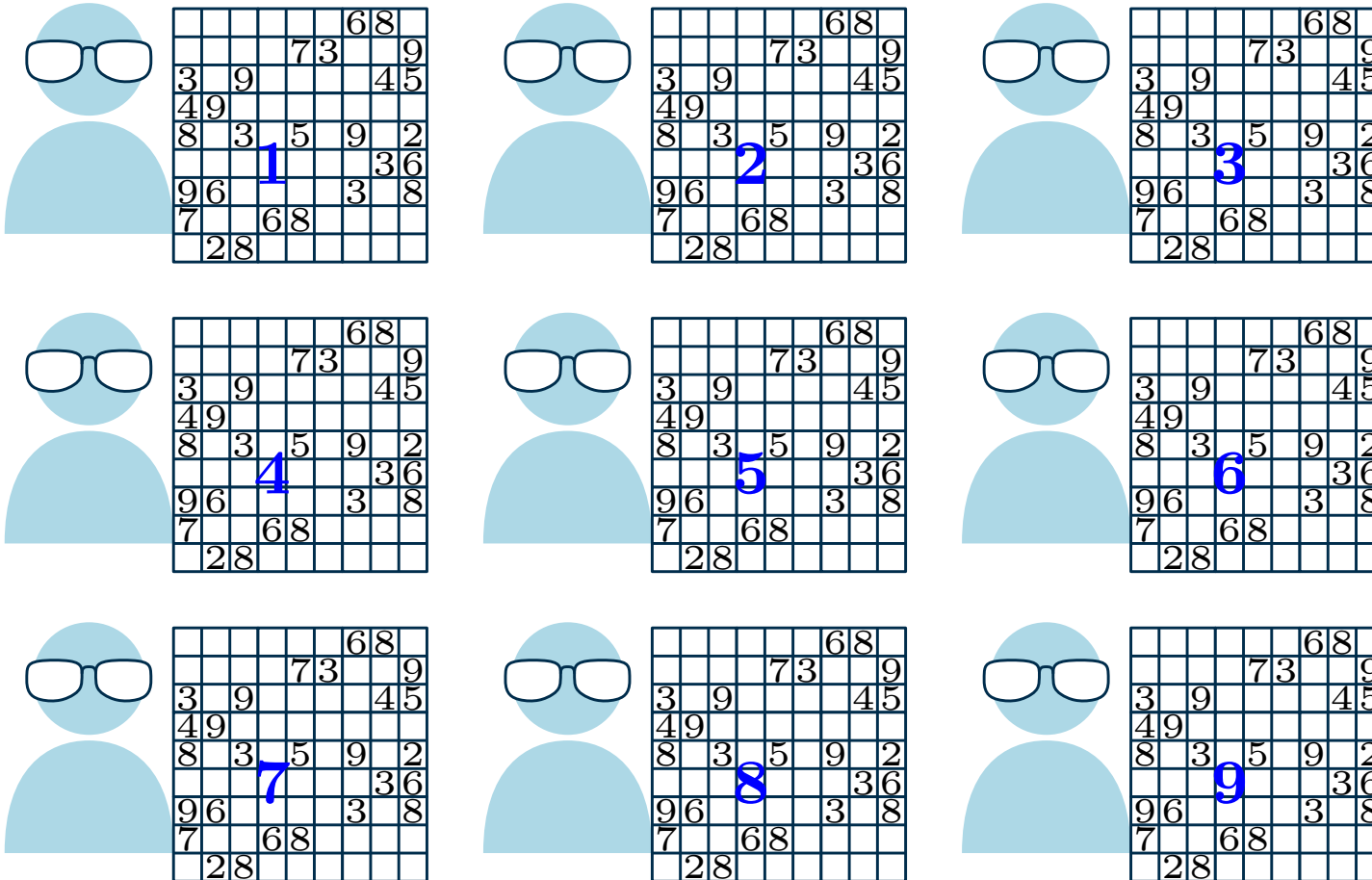
The Assembly of Nerds

- Complex and large logic puzzle
- n puzzle experts at your disposition
 - **anti-social**: work best if left undisturbed

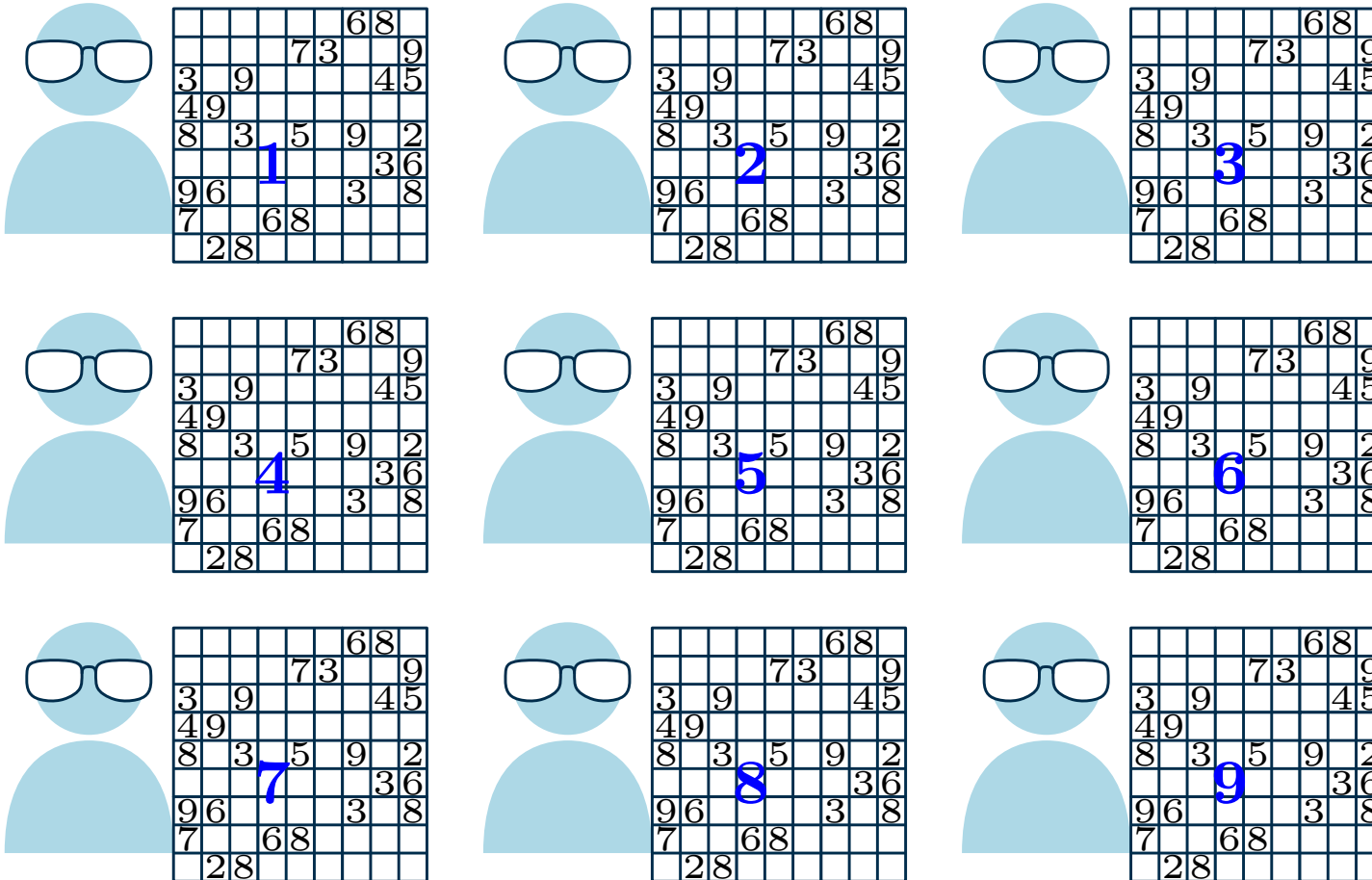
How do we employ and “orchestrate” our experts?



Approach I: Search Space Partitioning



Approach I: Search Space Partitioning



- Partition search space at some decisions
⇒ Independent subproblems

Explicit Partitioning

Böhm & Speckenmeyer (1994): 1st Parallel DPLL Implementation

Explicit Load Balancing

- Completely distributed (no leader / worker roles)
- A list of **partial assignments** is generated
- Each process receives the entire formula and **a few partial assignments**
- Each process can be worker or balancer:
 - **Worker:** solve or split the formula, use the partial assignments
 - **Balancer:** estimate workload, communicate, stop
- Switch to balancer whenever worker is finished

Explicit Partitioning

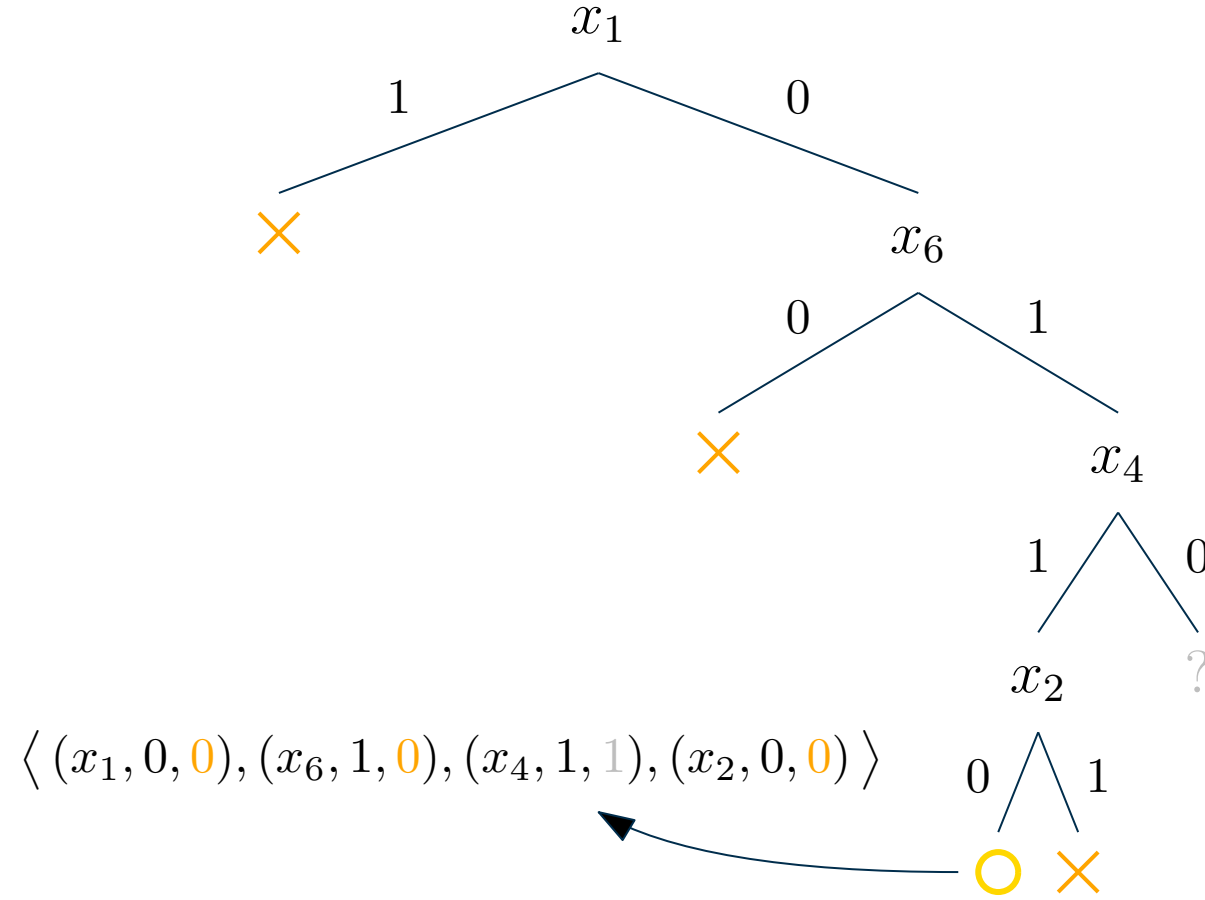
“**PSATO**: a Distributed Propositional Prover and its Application to Quasigroup Problems”, Zhang et al., 1996

Centralized leader-worker architecture

- Communication only between leader and workers
- Leader assigns partial assignments using **Guiding Path**
 - Each node in the search tree is **open** or **closed**
 - closed = branch is **explored** / **proven unsat**
 - Leader splits open nodes and assigns job to workers
- Workers return **Guiding Path** when terminated by leader
- Modern features of **fault tolerance**, **preemption** of solving tasks

Explicit Partitioning

Guiding Path: List of triples (variable, branch, open)



Explicit Partitioning Solvers

SATZ (Jurkowiak et al., 2001): *Work stealing* for workload balancing

- An idle worker *requests work* from the leader
- The leader *splits the work* of the most loaded worker
- The idle worker and most loaded worker get the parts

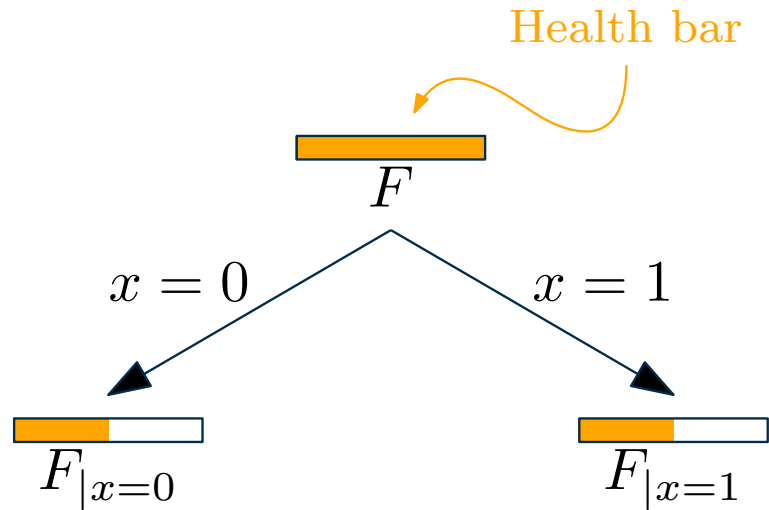
PaSAT (Blochinger et al., 2001)

- First parallel CDCL with *clause sharing*
- Similar to PSATO/SATZ: leader/worker, guiding path, work stealing

ySAT (Feldman et al., 2004)

- First *shared-memory parallel* solver
- Multi-core processors started to be popular
- uses same techniques as the previous solvers (guiding path etc.)

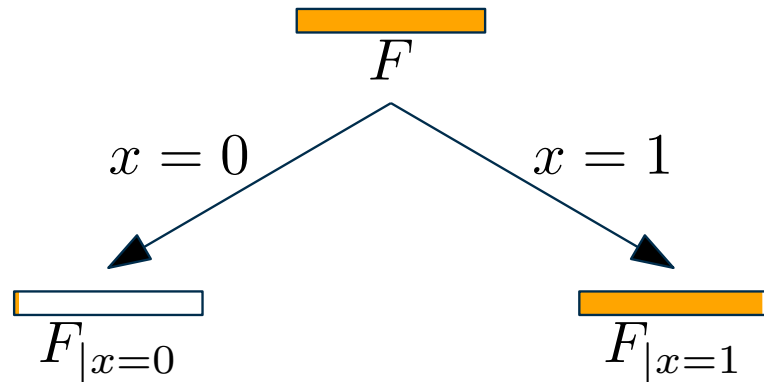
Problems with Partitioning



What we want: **Even splits**

- Split yields sub-formulas of **similar difficulty**
- **Balanced** partitioning of work
- Few or no dynamic (re-)balancing needed

Problems with Partitioning



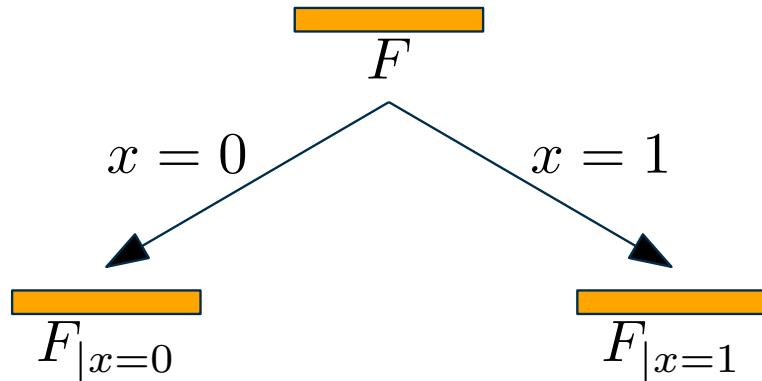
What we want: **Even splits**

- Split yields sub-formulas of **similar difficulty**
- **Balanced** partitioning of work
- Few or no dynamic (re-)balancing needed

Uneven splits

- One subformula is **trivial**, the other is **just as hard as F**
- **Ping-pong effect** for workers processing trivial formulae, communication / synchronization dominates run time

Problems with Partitioning



What we want: **Even splits**

- Split yields sub-formulas of **similar difficulty**
- Balanced partitioning of work
- Few or no dynamic (re-)balancing needed

Uneven splits

- One subformula is **trivial**, the other is **just as hard as F**
- **Ping-pong effect** for workers processing trivial formulae, communication / synchronization dominates run time

Bogus splits

- Both $F|_{x=0}$ and $F|_{x=1}$ are **just as hard as F**
- **Divide&Conquer** becomes **Multiply&Surrender!**

Cube and Conquer

The Cube&Conquer paradigm (Heule & Biere, 2011)

Generate a large amount (millions) of partial assignments (“**cubes**”) and randomly assign them to workers.

Cube and Conquer

The Cube&Conquer paradigm (Heule & Biere, 2011)

Generate a large amount (millions) of partial assignments (“**cubes**”) and randomly assign them to workers.

- Unlikely that any of the workers will run out tasks
⇒ Hope of good load balancing in practice
- Partial assignments are generated using a look-ahead solver (breadth-first search up to a limited depth)
- Best performance mostly with problem-specific decision heuristics

Cube and Conquer

The Cube&Conquer paradigm (Heule & Biere, 2011)

Generate a large amount (millions) of partial assignments (“**cubes**”) and randomly assign them to workers.

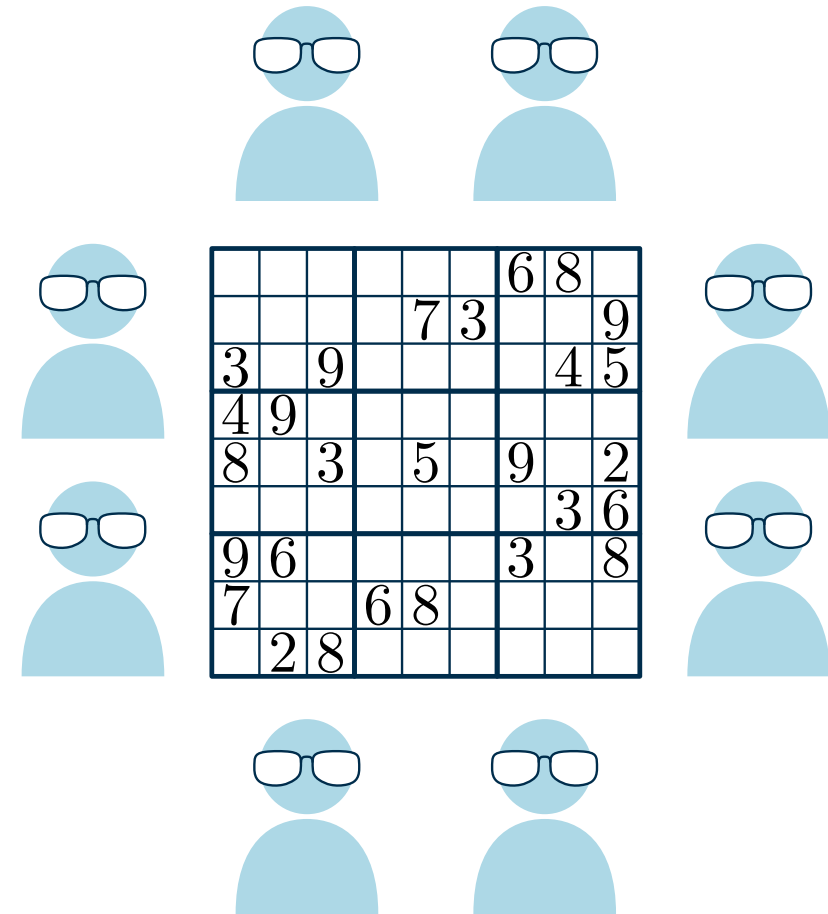
- Unlikely that any of the workers will run out tasks
⇒ Hope of good load balancing in practice
- Partial assignments are generated using a look-ahead solver (breadth-first search up to a limited depth)
- Best performance mostly with problem-specific decision heuristics
- State-of-the-art for hard combinatorial problems
 - Used to solve the “Pythagorean Triples” problem (~200TB proof)
 - ... or more recently “Schur Number 5” (~2PB proof)
- Examples: March (Heule) + iLingeling (Biere) introduced in 2011; Treengeling (Biere)

Parallel Portfolios: An analogy

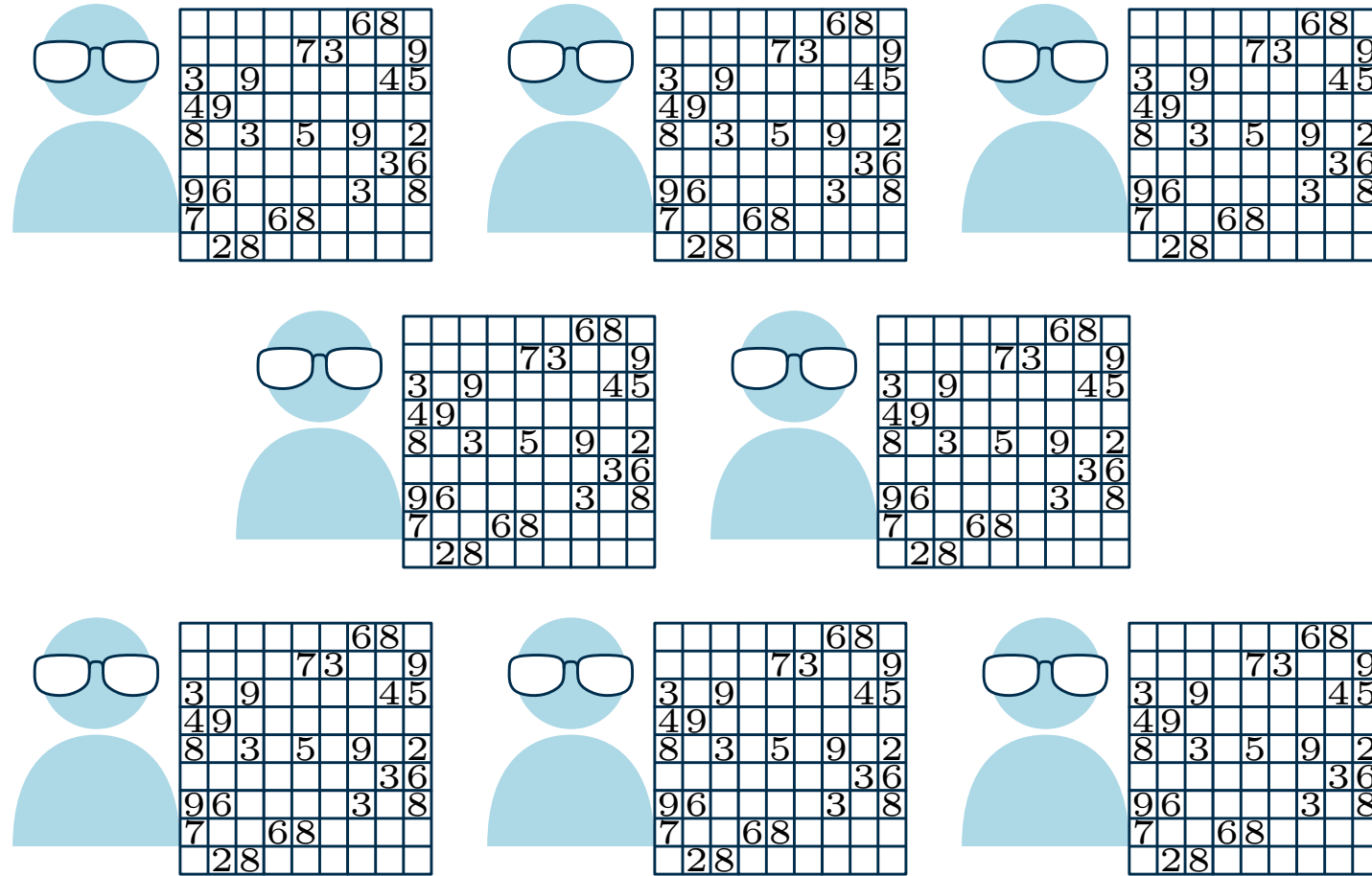
The Assembly of Nerds

- Complex and large logic puzzle
- n puzzle experts at your disposition
 - individual mindsets, approaches, strengths & weaknesses
 - **anti-social**: work best if left undisturbed

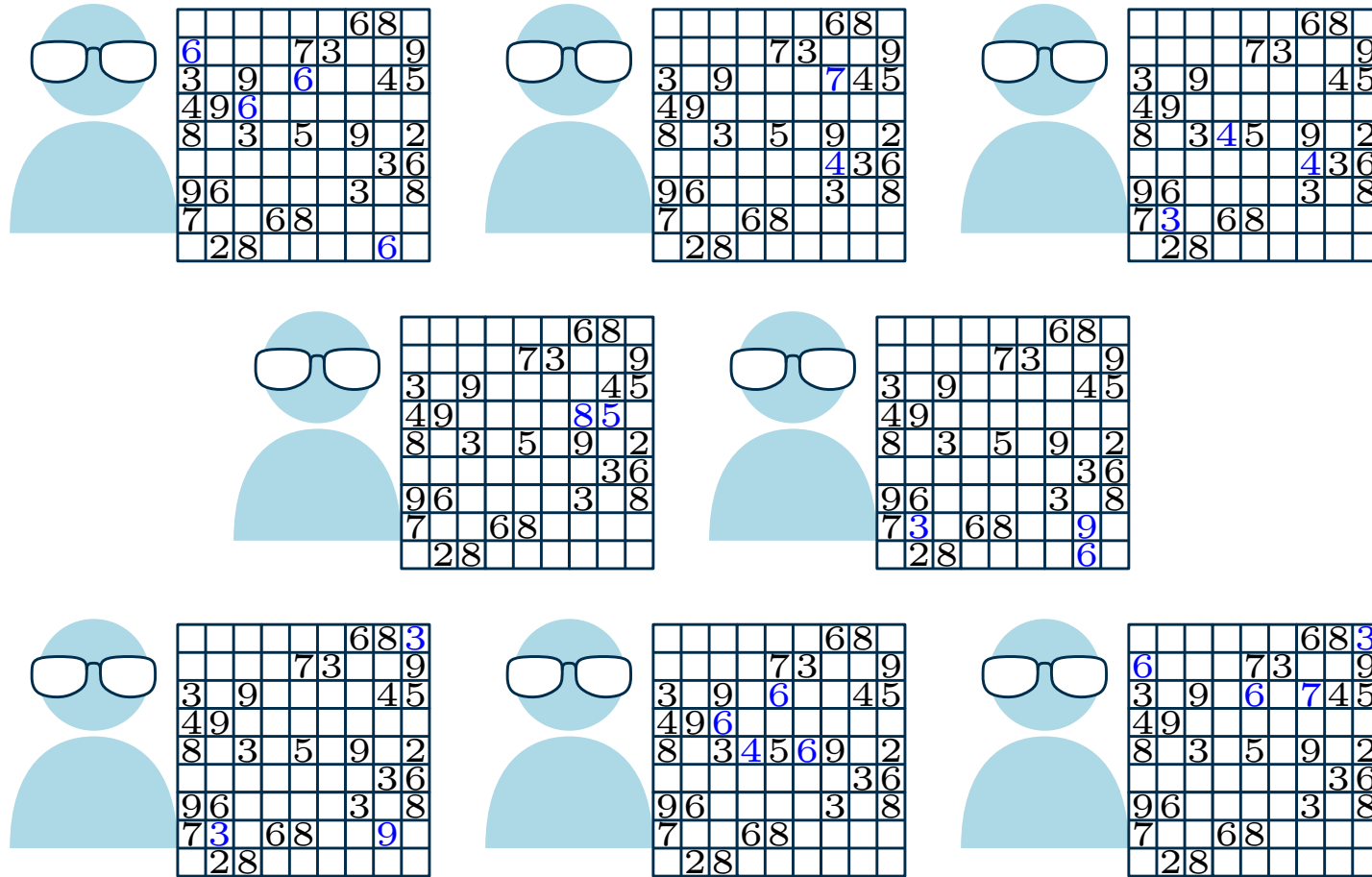
How do we employ and “orchestrate” our experts?



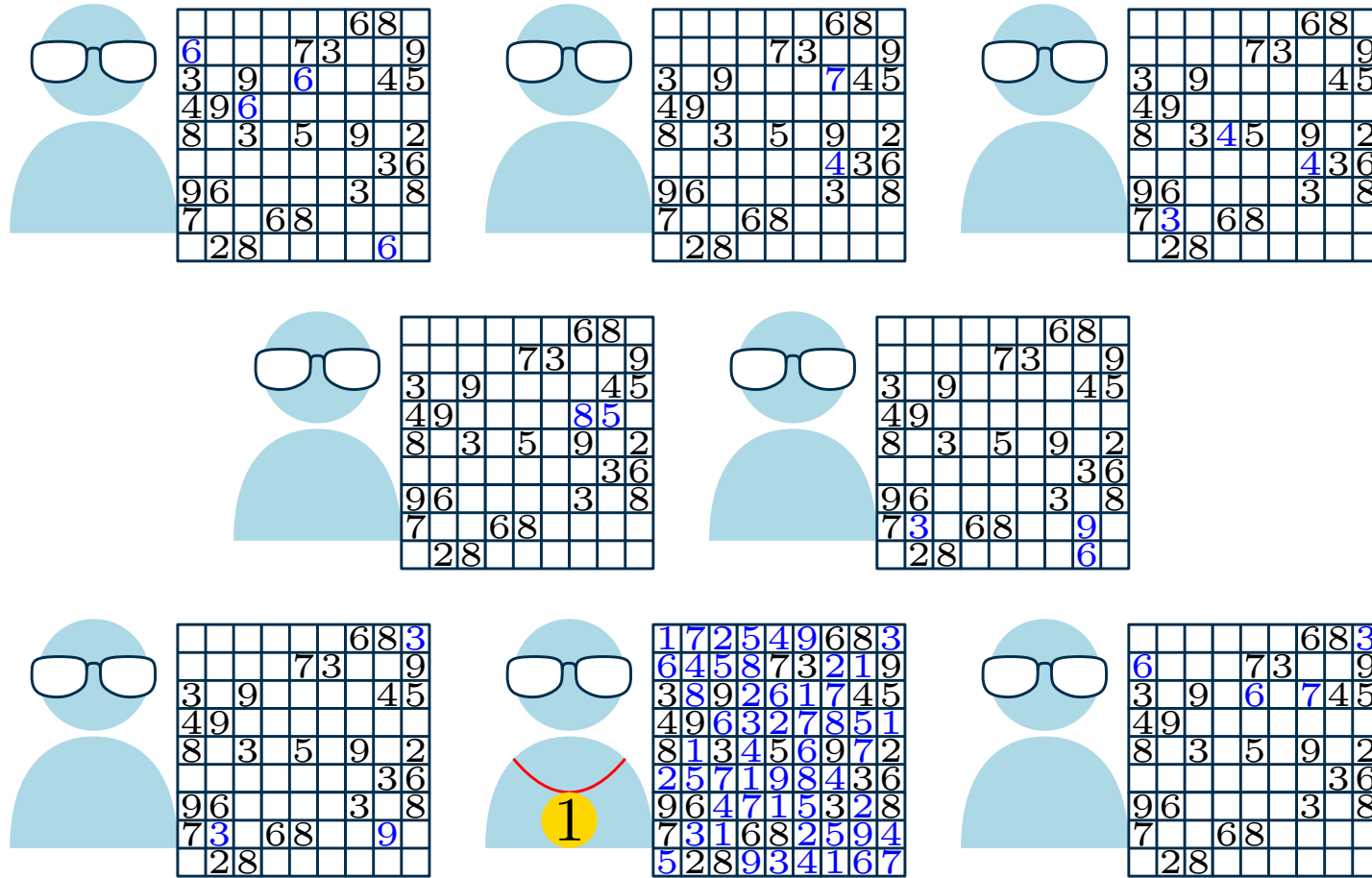
Approach II: Pure Portfolio



Approach II: Pure Portfolio



Approach II: Pure Portfolio



Pure Portfolio: Oracle view vs. Speedup view

Virtual Best Solver (VBS) / Oracle

Consider n algorithms $A_1, \dots, A_n$ where for each input x , algorithm A_i has run time $T_{A_i}(x)$.
The Virtual Best Solver (VBS) for $A_1, \dots, A_n$ has run time $T^*(x) = \min\{T_{A_1}(x), \dots, T_{A_n}(x)\}$.

Pure Portfolio: Oracle view vs. Speedup view

Virtual Best Solver (VBS) / Oracle

Consider n algorithms $A_1, \dots, A_n$ where for each input x , algorithm A_i has run time $T_{A_i}(x)$.
The **Virtual Best Solver** (VBS) for $A_1, \dots, A_n$ has run time $T^*(x) = \min\{T_{A_1}(x), \dots, T_{A_n}(x)\}$.

Optimist: A pure portfolio **simulates** the VBS using parallel processing!

- On idealized hardware, we “select” best sequential solver for each instance

Pure Portfolio: Oracle view vs. Speedup view

Virtual Best Solver (VBS) / Oracle

Consider n algorithms $A_1, \dots, A_n$ where for each input x , algorithm A_i has run time $T_{A_i}(x)$. The Virtual Best Solver (VBS) for $A_1, \dots, A_n$ has run time $T^*(x) = \min\{T_{A_1}(x), \dots, T_{A_n}(x)\}$.

Optimist: A pure portfolio simulates the VBS using parallel processing!

- On idealized hardware, we “select” best sequential solver for each instance

Parallel speedup

Given parallel algorithm P and input x , the speedup of P is defined as $s_P(x) = T_Q(x)/T_P(x)$ where Q is the best available sequential algorithm.

Pure Portfolio: Oracle view vs. Speedup view

Virtual Best Solver (VBS) / Oracle

Consider n algorithms $A_1, \dots, A_n$ where for each input x , algorithm A_i has run time $T_{A_i}(x)$.
The **Virtual Best Solver** (VBS) for $A_1, \dots, A_n$ has run time $T^*(x) = \min\{T_{A_1}(x), \dots, T_{A_n}(x)\}$.

Optimist: A pure portfolio **simulates** the VBS using parallel processing!

- On idealized hardware, we “select” best sequential solver for each instance

Parallel speedup

Given parallel algorithm P and input x , the **speedup** of P is defined as $s_P(x) = T_Q(x)/T_P(x)$ where Q is the **best available sequential algorithm**.

Pessimist: A pure portfolio **never achieves actual speedups!**

- There is always a sequential algorithm performing **at least as well**
- Consequence: **Not resource efficient, not scalable**

Pure SAT Portfolios

ppfolio: Winner of Parallel Track in the 2011 SAT Competition

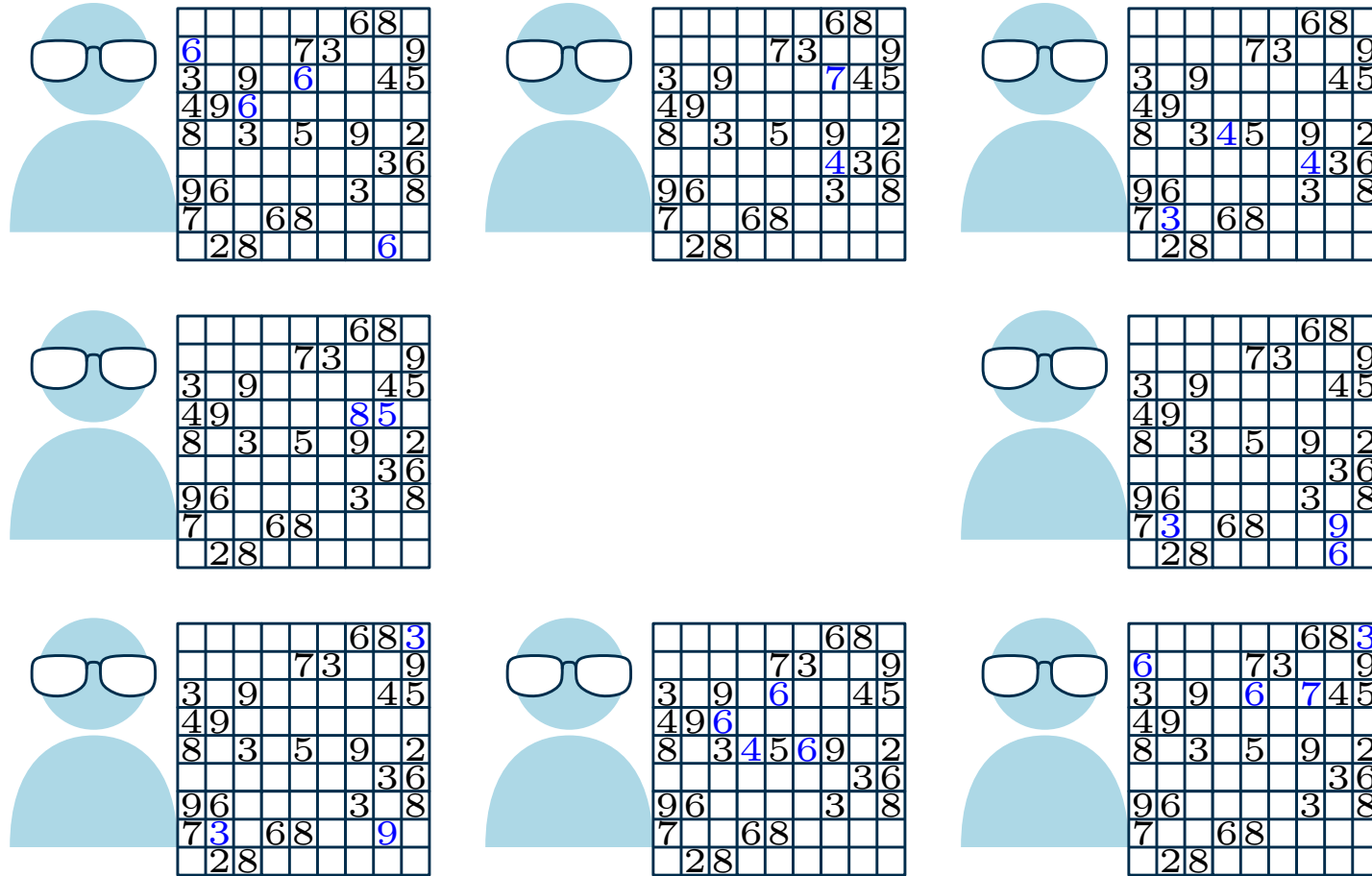
- Just a bash script combining the best sequential solvers from 2010:
~\$./solver1 f.cnf & ./solver2 f.cnf & ./solver3 f.cnf & ./solver4 f.cnf
- Bits by O. Roussel, the author of ppfolio:
 - “*by definition the best solver on Earth*”
 - “*probably the laziest and most stupid solver ever written*”

Pure SAT Portfolios

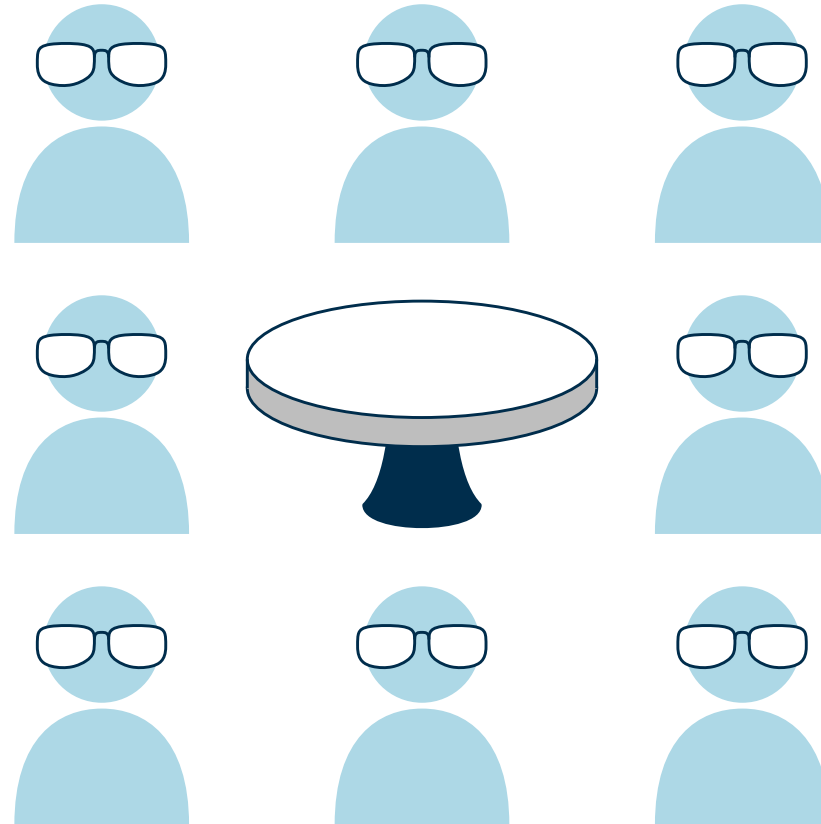
ppfolio: Winner of Parallel Track in the 2011 SAT Competition

- Just a bash script combining the best sequential solvers from 2010:
~\$./solver1 f.cnf & ./solver2 f.cnf & ./solver3 f.cnf & ./solver4 f.cnf
- Bits by O. Roussel, the author of ppfolio:
 - “*by definition the best solver on Earth*”
 - “*probably the laziest and most stupid solver ever written*”
- Rationale: Different solvers are designed differently, excel on different instances
 - hope of orthogonal search behavior
- Pure portfolios no longer permitted in SAT Competitions

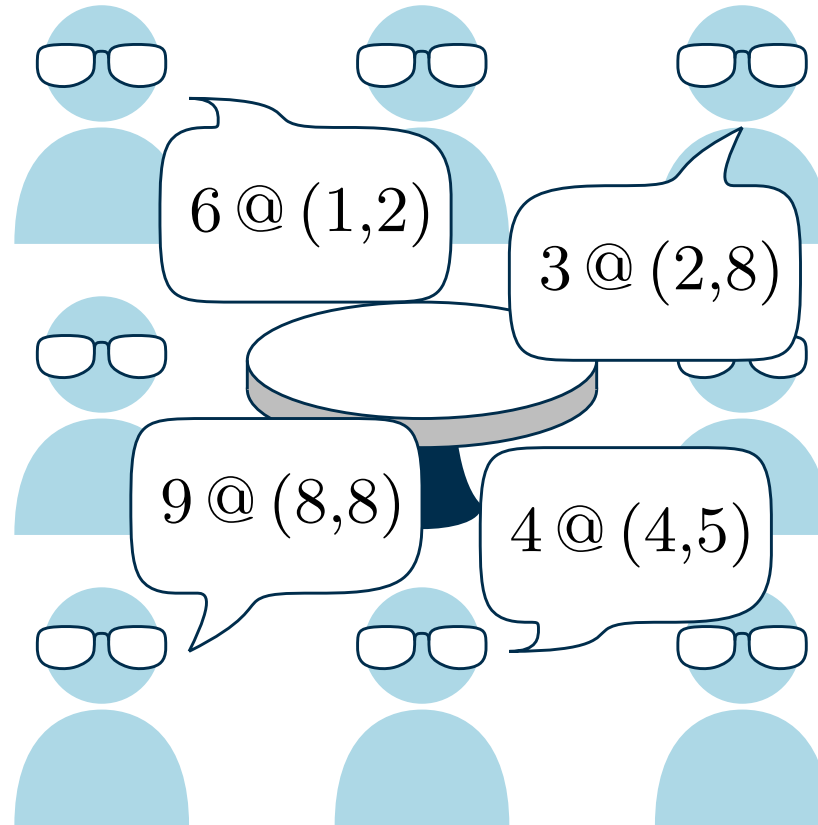
Approach II+: Cooperative Portfolio



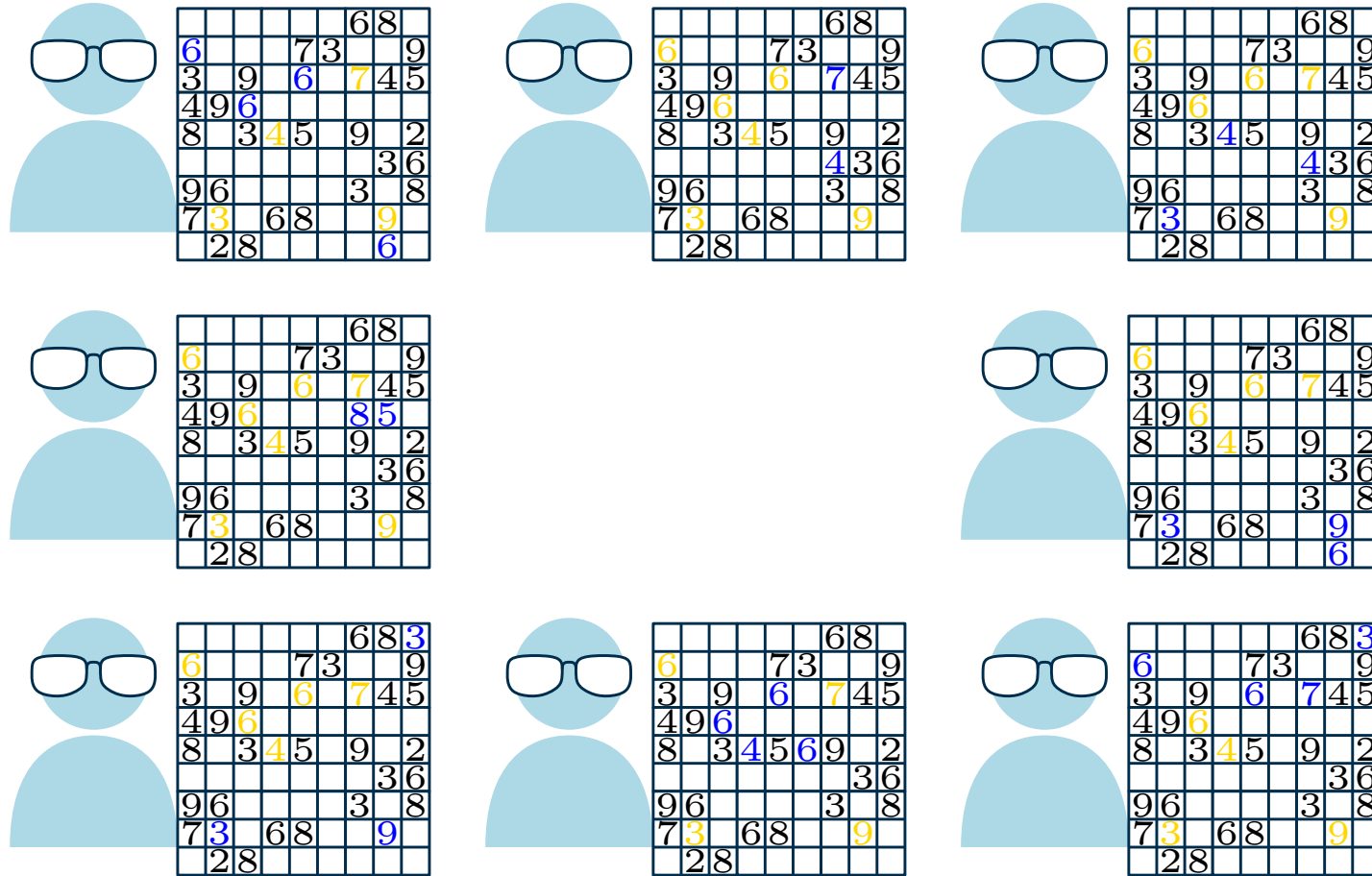
Approach II+: Cooperative Portfolio



Approach II+: Cooperative Portfolio



Approach II+: Cooperative Portfolio



Cooperative Portfolio

Assembly of Nerds, enhanced

- The experts periodically gather for **brief standup meetings**
- Via some protocol, the experts exchange the **most valuable insights** gained since the last meeting
- Solving continues — each expert may use the shared insights **at their own discretion**

Equivalent to “insights” in SAT solving:

Cooperative Portfolio

Assembly of Nerds, enhanced

- The experts periodically gather for **brief standup meetings**
- Via some protocol, the experts exchange the **most valuable insights** gained since the last meeting
- Solving continues — each expert may use the shared insights **at their own discretion**

Equivalent to “insights” in SAT solving: **learnt (conflict) clauses**

- Explored branch of search space — **safe to prune**
- Potential step for deriving **unsatisfiability**
- Result: **Parallel search space pruning procedure**

Cooperative Portfolio

Assembly of Nerds, enhanced

- The experts periodically gather for brief standup meetings
- Via some protocol, the experts exchange the most valuable insights gained since the last meeting
- Solving continues — each expert may use the shared insights at their own discretion

Equivalent to “insights” in SAT solving: **learnt (conflict) clauses**

- Explored branch of search space — safe to prune
- Potential step for deriving unsatisfiability
- Result: Parallel search space pruning procedure

Combination of portfolio idea + clause exchanges: **Clause sharing portfolio solvers**

Clause Sharing Portfolios: Design Space

Portfolio considerations

- Which sequential solvers to employ?
- How to diversify solvers?
 - different search algorithms, selection heuristics, restart intervals, ...
 - different random seeds, initial phases, input permutations, ...

```
void Cadical::diversify(int seed) {  
    solver->set(name: "seed", val: seed);  
    switch (getDiversificationIndex() % getNumOriginalDiversifications()) {  
    case 0: okay = solver->set(name: "phase", val: 0); break;  
    case 1: okay = solver->configure("sat"); break;  
    case 2: okay = solver->set(name: "elim", val: 0); break;  
    case 3: okay = solver->configure("unsat"); break;  
    case 4: okay = solver->set(name: "condition", val: 1); break;  
    case 5: okay = solver->set(name: "walk", val: 0); break;  
    case 6: okay = solver->set(name: "restartint", val: 100); break;  
    case 7: okay = solver->set(name: "cover", val: 1); break;  
    case 8: okay = solver->set(name: "shuffle", val: 1) && solver->set(name: "  
    case 9: okay = solver->set(name: "inprocessing", val: 0); break;
```

Clause Sharing Portfolios: Design Space

Portfolio considerations

- Which sequential solvers to employ?
- How to diversify solvers?
 - different search algorithms, selection heuristics, restart intervals, ...
 - different random seeds, initial phases, input permutations, ...

Clause exchange considerations

- How often to share? (immediate/eager? delayed/lazy? periodic?)
- How many clauses to share? (fixed volume? fixed quality criteria?)
- Which clauses to share? (shortest? lowest LBD?)
- How to implement sharing? (all-to-all? leader-worker? some communication graph?)

Early Clause Sharing Portfolios

ManySAT (Hamadi, Jabbour, and Sais 2009)

- Hand-crafted diversification of **four solver configurations**
 - Restart policy, variable + polarity selection heuristic, ...
- Eager exchange of clauses of length ≤ 8 via **lockless queues**

Early Clause Sharing Portfolios

ManySAT (Hamadi, Jabbour, and Sais 2009)

- Hand-crafted diversification of **four solver configurations**
 - Restart policy, variable + polarity selection heuristic, ...
- Eager exchange of clauses of length ≤ 8 via **lockless queues**

Plingeling (Biere 2010)

- Portfolio over **Lingeling configurations** (shared-memory parallelism)
- Lazy exchange of information over “boss thread”
 - 2010: **Unit clauses** only
 - 2011: Unit clauses + **equivalences**
 - Since 2013: Unit clauses + equivalences + **clauses of length ≤ 40 , LBD ≤ 8**
- **Best parallel solver** for many years

Massively parallel hardware?

Distributed computing, High-Performance Computing (HPC)

In distributed computing, several machines (with **no shared main memory**) run together. On each machine we run a number of processes, each of which runs on a number of cores. Processes commonly communicate by exchanging messages.



SuperMUC-NG: 6 336 nodes × 48 cores

Massively parallel hardware?

Distributed computing, High-Performance Computing (HPC)

In distributed computing, several machines (with **no shared main memory**) run together. On each machine we run a number of processes, each of which runs on a number of cores. Processes commonly communicate by exchanging messages.



SuperMUC-NG: 6 336 nodes × 48 cores

Large distributed systems (hundreds to thousands of cores) impose new requirements, challenges:

- No shared memory — communication protocols required
- Diminishing returns due to **exhausted diversification** of solvers

Massively parallel hardware?

Distributed computing, High-Performance Computing (HPC)

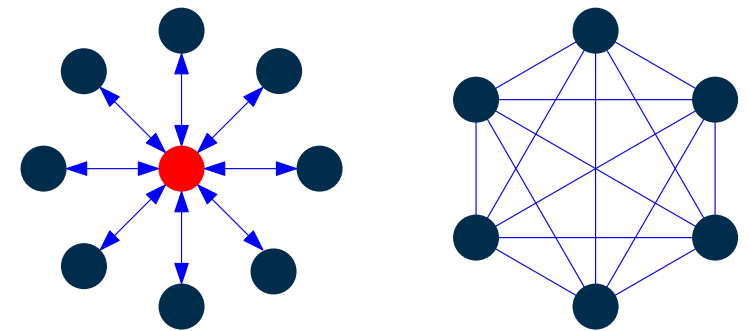
In distributed computing, several machines (with **no shared main memory**) run together. On each machine we run a number of **processes**, each of which runs on a number of **cores**. Processes commonly communicate by **exchanging messages**.



SuperMUC-NG: 6 336 nodes $\times$ 48 cores

Large distributed systems (hundreds to thousands of cores) impose new requirements, challenges:

- No shared memory — **communication protocols** required
- Diminishing returns due to **exhausted diversification** of solvers
- Some exchange schemes are **conceptually not scalable**
 - “**Star graph**”: Master process collects, serves all exported clauses
 - **Naïve (quadratic) all-to-all exchange** of clauses
- HPC schedulers, administrators, committees are used to **regular, easily parallelizable code** with **near-linear scaling**



Massively parallel SAT portfolio

HordeSat (Balyo, Sanders, Sinz 2015)

- Decentralization: No single leader node / process
- Two-level (“hybrid”) parallelization
 - One or several processes on each machine
 - Multiple solver threads (+ communication thread) on each process

Massively parallel SAT portfolio

HordeSat (Balyo, Sanders, Sinz 2015)

- Decentralization: No single leader node / process
- Two-level (“hybrid”) parallelization
 - One or several processes on each machine
 - Multiple solver threads (+ communication thread) on each process
- Diversification options:
 - Native diversification (set of hand-crafted solver configurations)
 - Modifying some initial variable phases
 - Random seeds
- Periodic all-to-all clause exchange

Clause Exchange in HordeSat

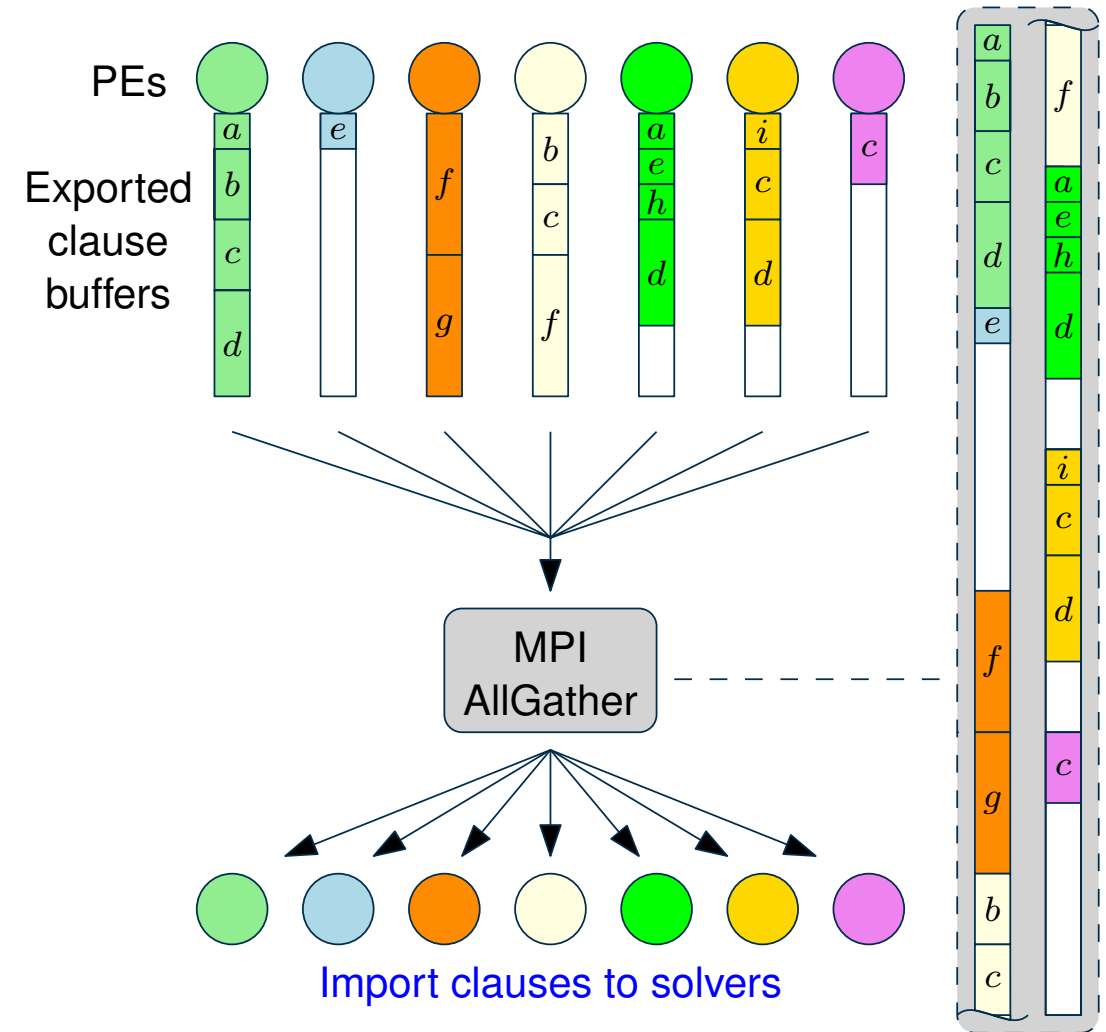
HordeSat's sharing logic

- Only clauses with $LBD \leq 2$ are considered for sharing
 - Constraint is lifted successively if processes under-produce
- Solver threads write eligible clauses into shared buffer

Clause Exchange in HordeSat

HordeSat's sharing logic

- Only clauses with $LBD \leq 2$ are considered for sharing
 - Constraint is lifted successively if processes under-produce
 - Solver threads write eligible clauses into shared buffer
 - Each process' **best clauses** are shared with everyone
 - Limited to 1500 clause literals per process
- ⇒ Concatenation of p produced clause buffers
- Approximate, post-hoc **filtering** of clauses



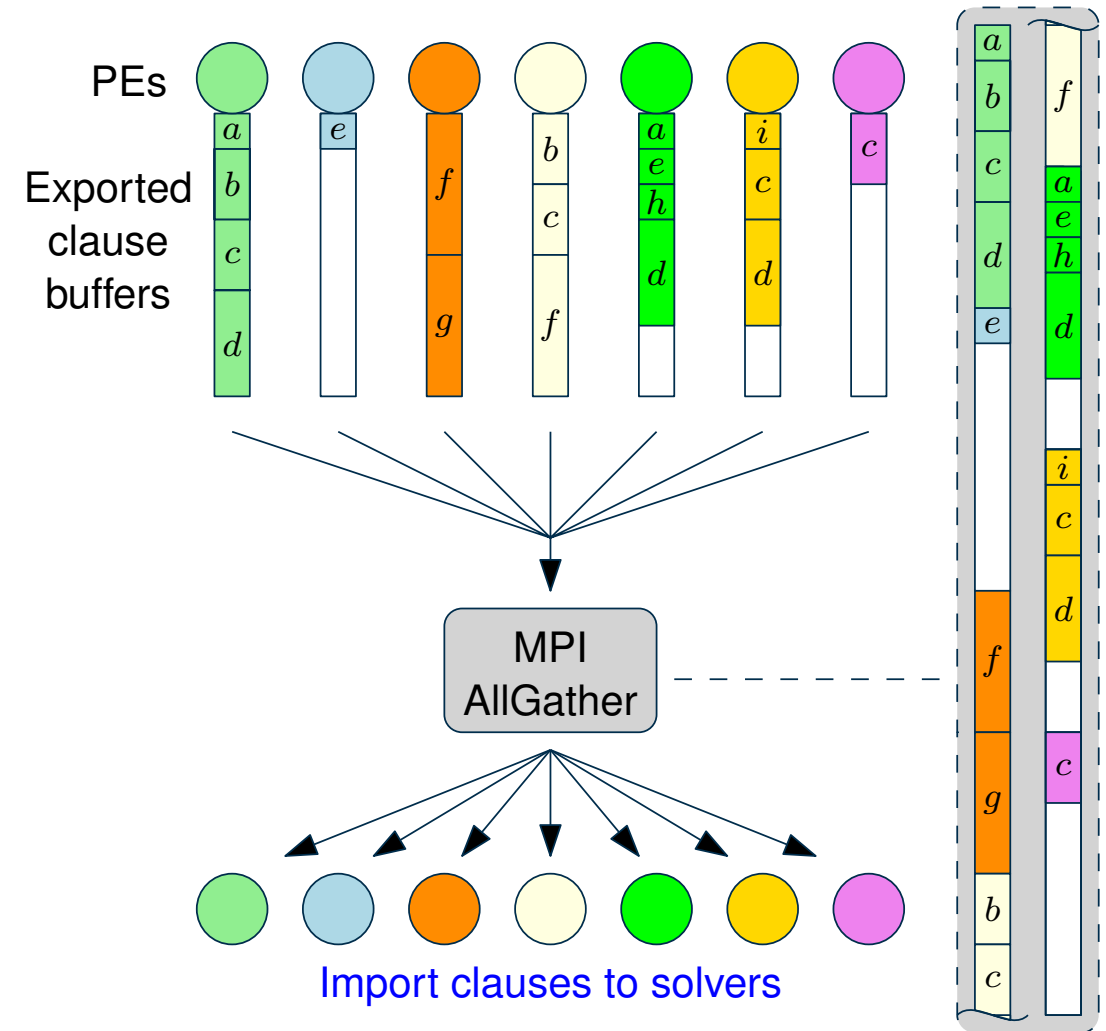
Clause Exchange in HordeSat

HordeSat's sharing logic

- Only clauses with $LBD \leq 2$ are considered for sharing
 - Constraint is lifted successively if processes under-produce
 - Solver threads write eligible clauses into shared buffer
 - Each process' **best clauses** are shared with everyone
 - Limited to 1500 clause literals per process
- ⇒ Concatenation of p produced clause buffers
- Approximate, post-hoc **filtering** of clauses

Issues:

- Many ("high" LBD) clauses are not shared but **discarded**
- **"Holes"** in buffer carrying no information
- **Duplicate** clauses
- Buffer grows **proportionally** with # proc.
 - ⇒ **Bottleneck** w.r.t. communication, local work



HordeSat: Results

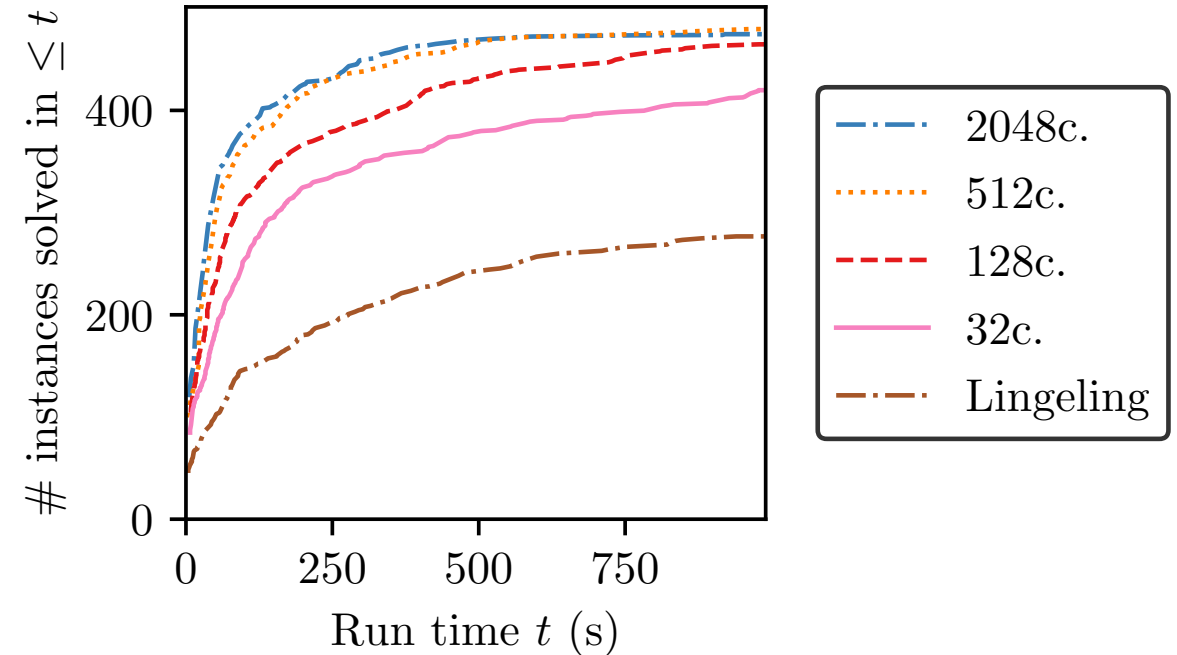
- Super-linear speedups for individual instances
= speedup $> c$ on c cores! How?

HordeSat: Results

- Super-linear speedups for individual instances
= speedup $> c$ on c cores! **How?**
 - SAT: “NP luck” – some solver got lucky
 - UNSAT: distributed memory accommodates more clauses than any sequential solver
 - General: sequential schedule of parallel algorithm may **outperform** sequential algorithm!

HordeSat: Results

- Super-linear speedups for individual instances
= speedup $> c$ on c cores! How?
 - SAT: “NP luck” – some solver got lucky
 - UNSAT: distributed memory accommodates more clauses than any sequential solver
 - General: sequential schedule of parallel algorithm may outperform sequential algorithm!
- Median speedup: 3 at 16 cores, 11.5 at 512 cores
 - Efficiency: $11.5/512 \approx 2.2\%$
 - Deploying HordeSat is often not worth it
- No improvement beyond ≈ 500 cores



Data extracted from HordeSat paper

From HordeSat to MallobSat

Research Question

How can we improve performance, (resource-)efficiency, and average response times of SAT solving in modern distributed environments?

From HordeSat to MallobSat

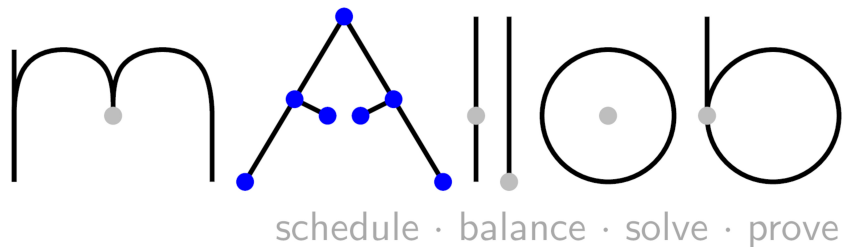
Research Question

How can we improve performance, (resource-)efficiency, and average response times of SAT solving in modern distributed environments?

Result: Mallob

Mallob is a platform for SAT solving (*and other NP-hard problems*) with:

- multi-user, on-demand, malleable scheduling and solving of many problems at once
- distributed SAT engine **MallobSat**: the HordeSat paradigm re-engineered and made efficient
- state-of-the-art SAT performance from dozens to thousands of cores

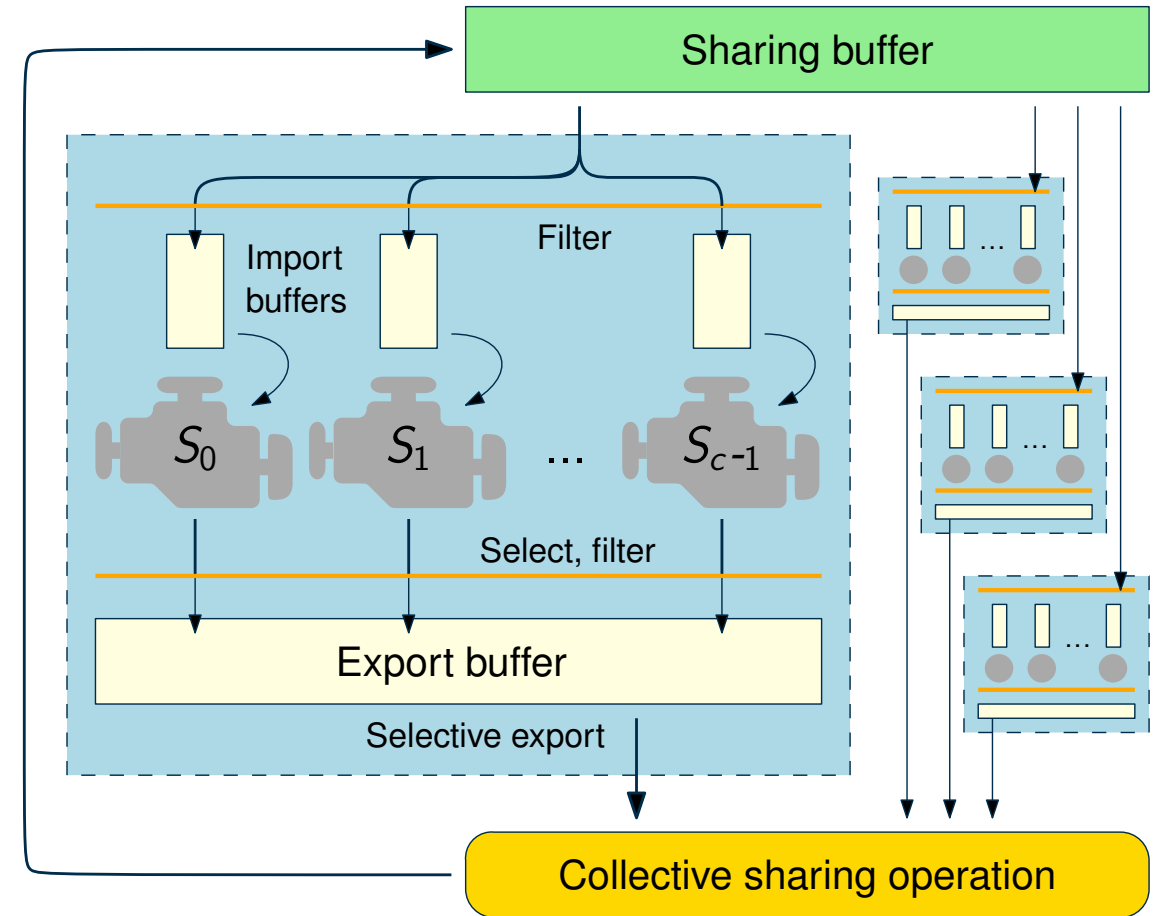


<https://satres.kikit.kit.edu/research/mallob>

Design Decisions of MallobSat

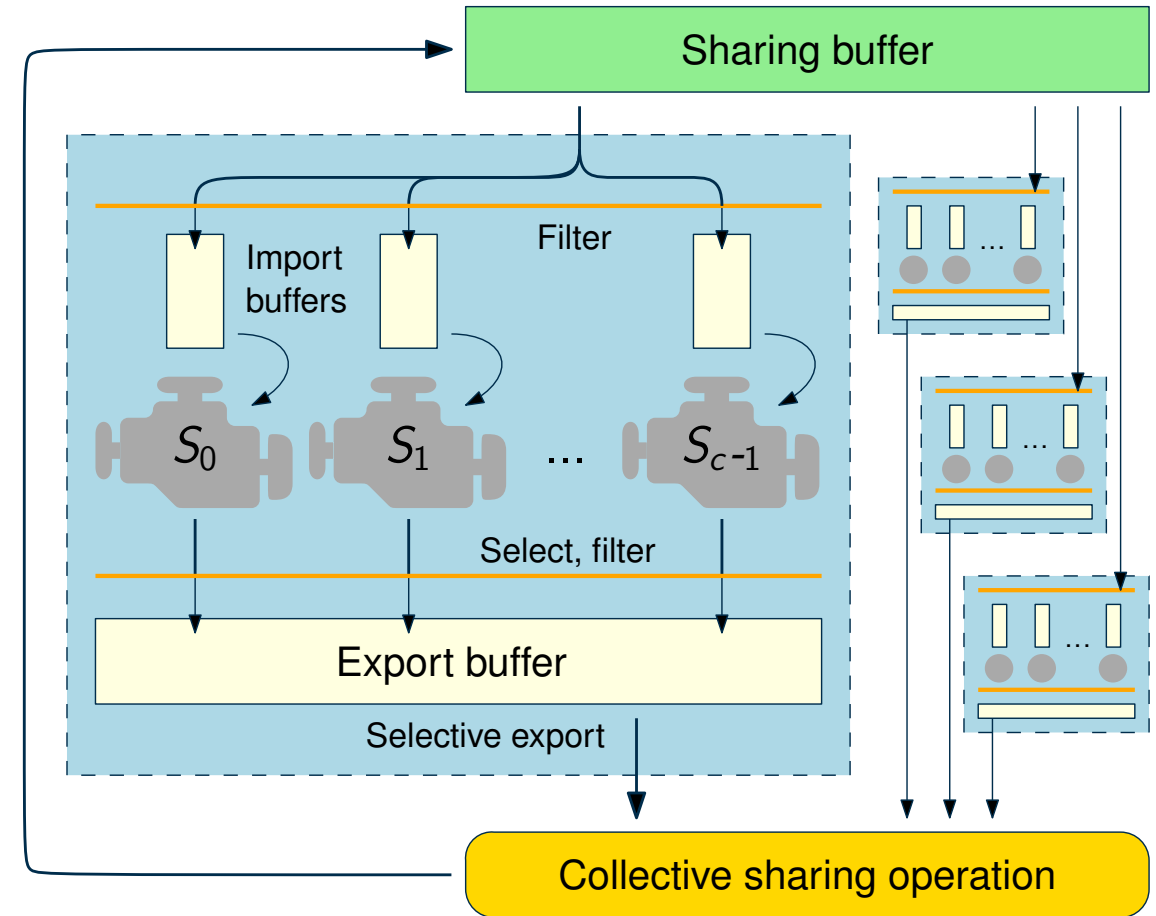
■ **Fundament:** HORDESAT

- Two-level hybrid parallelization
- Periodic all-to-all clause sharing



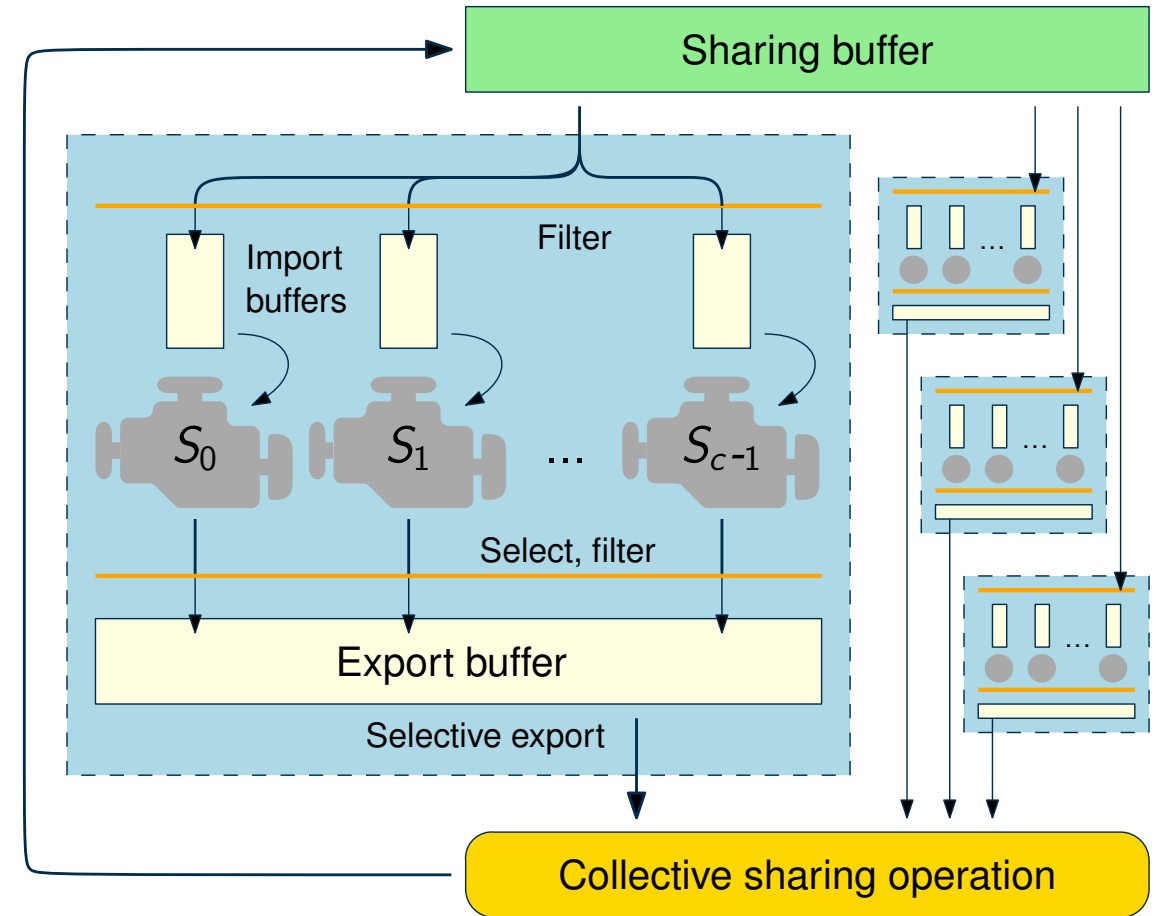
Design Decisions of MallobSat

- **Fundament:** HORDESAT
 - Two-level hybrid parallelization
 - Periodic all-to-all clause sharing
- Fix a certain sharing volume, spend it on the globally most useful **distinct** clauses



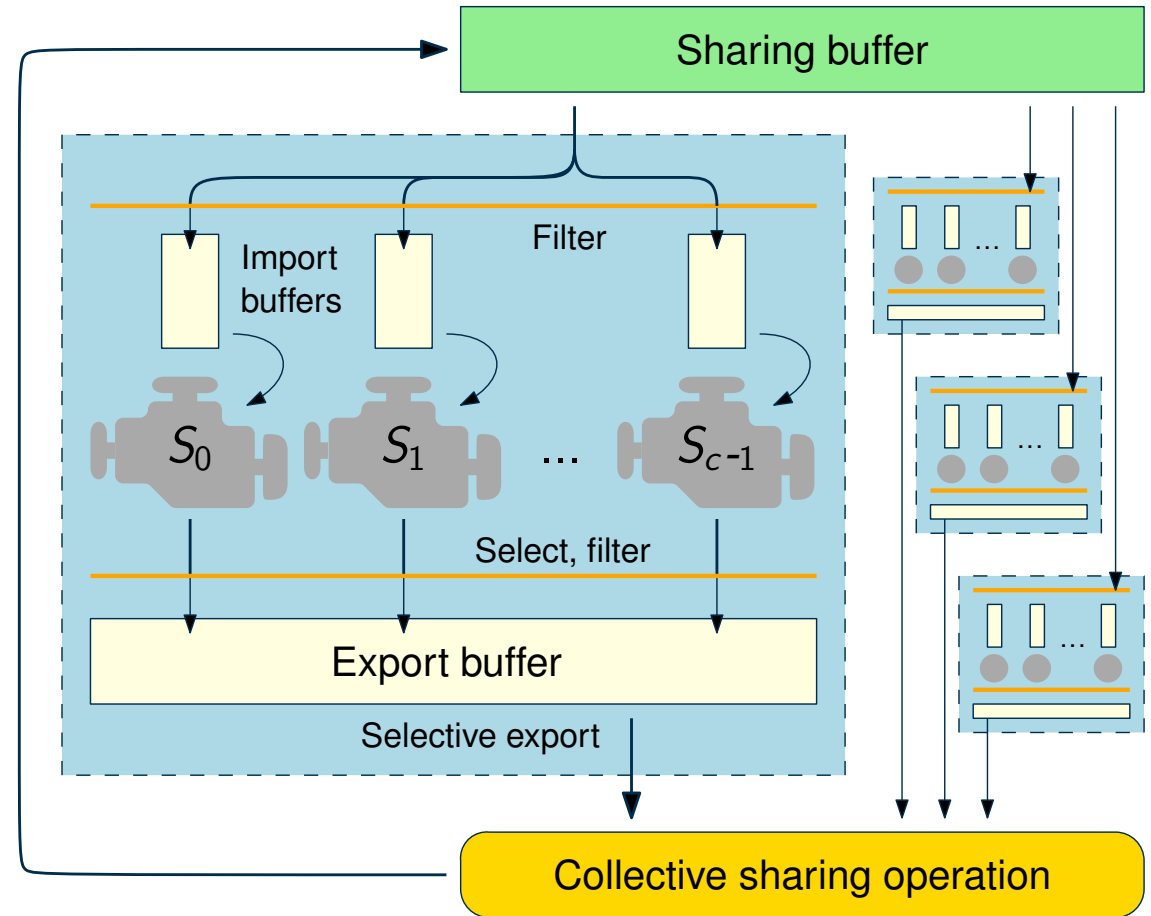
Design Decisions of MallobSat

- **Fundament:** HORDESAT
 - Two-level hybrid parallelization
 - Periodic all-to-all clause sharing
- Fix a certain sharing volume, spend it on the globally most useful **distinct** clauses
- Prioritize clauses by clause length over **LBD**
 - At all stages! Also in export / import buffering



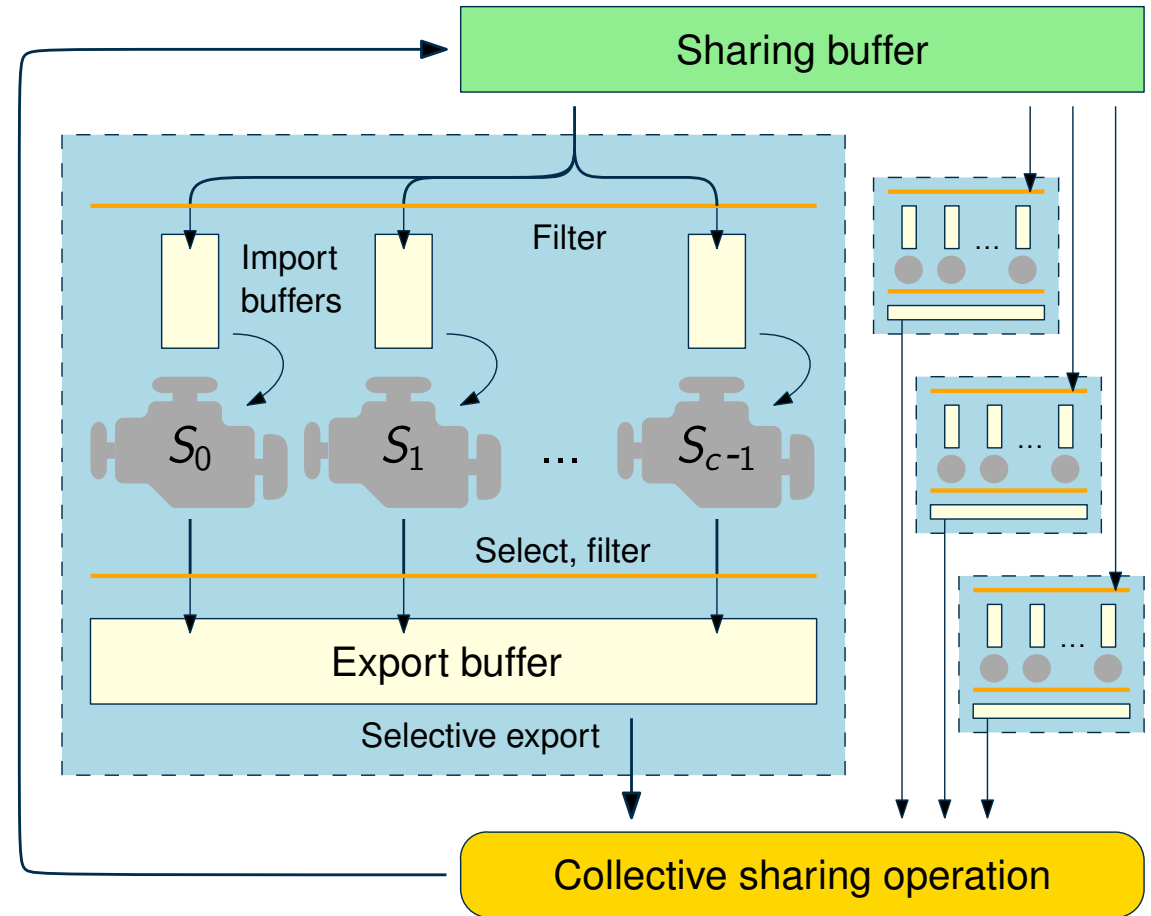
Design Decisions of MallobSat

- **Fundament:** HORDESAT
 - Two-level hybrid parallelization
 - Periodic all-to-all clause sharing
- Fix a certain sharing volume, spend it on the globally most useful **distinct** clauses
- Prioritize clauses by clause length over **LBD**
 - At all stages! Also in export / import buffering
- Minimize clause turnaround times



Design Decisions of MallobSat

- **Fundament:** HORDESAT
 - Two-level hybrid parallelization
 - Periodic all-to-all clause sharing
- Fix a certain sharing volume, spend it on the globally most useful **distinct** clauses
- Prioritize clauses by clause length over **LBD**
 - At all stages! Also in export / import buffering
- Minimize clause turnaround times
- Support **fluctuating workers** (*malleability*)

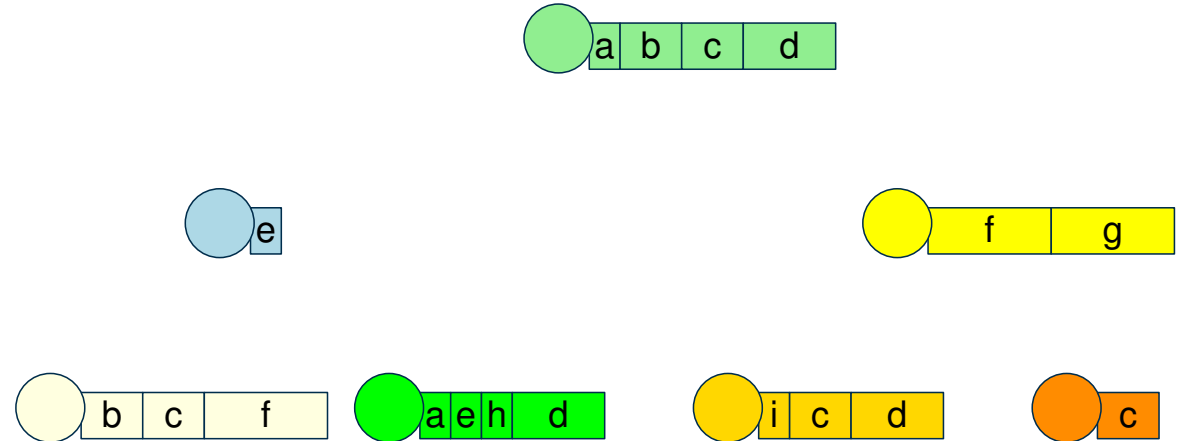


Clause Exchange in Mallob

Custom collective operation [SAT'21]

- Aggregate information along binary tree of processors
- Detect duplicates during merge
- Result is of compact shape
- Sublinear buffer size growth:
Discard longest clauses as necessary

1.

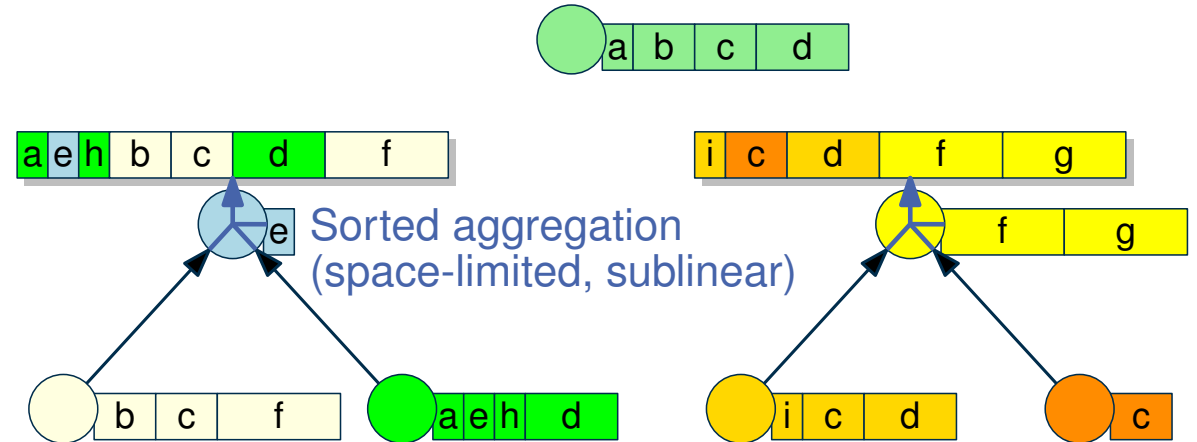


Clause Exchange in Mallob

Custom collective operation [SAT'21]

- Aggregate information along binary tree of processors
- Detect duplicates during merge
- Result is of compact shape
- Sublinear buffer size growth:
Discard longest clauses as necessary

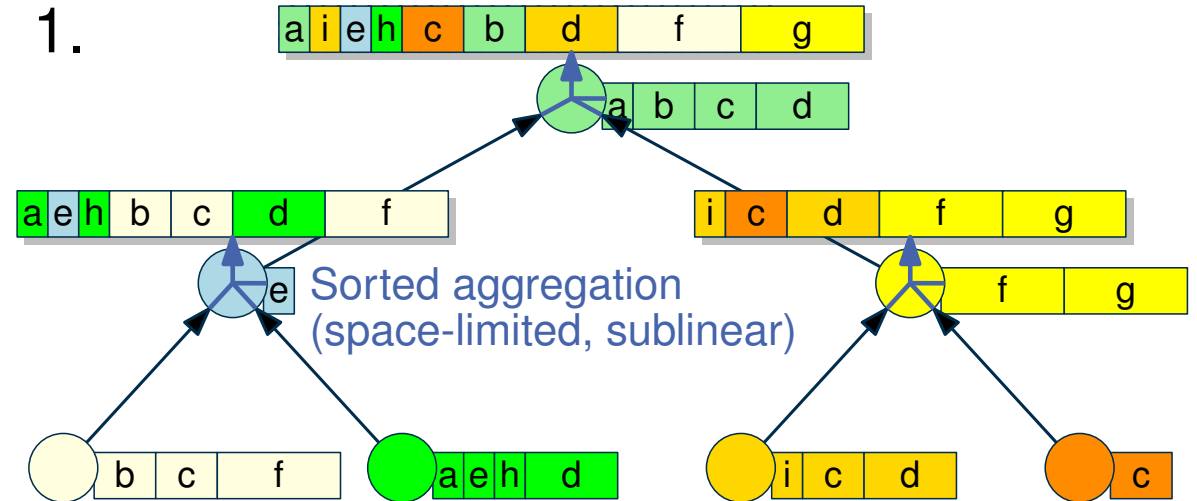
1.



Clause Exchange in Mallob

Custom collective operation [SAT'21]

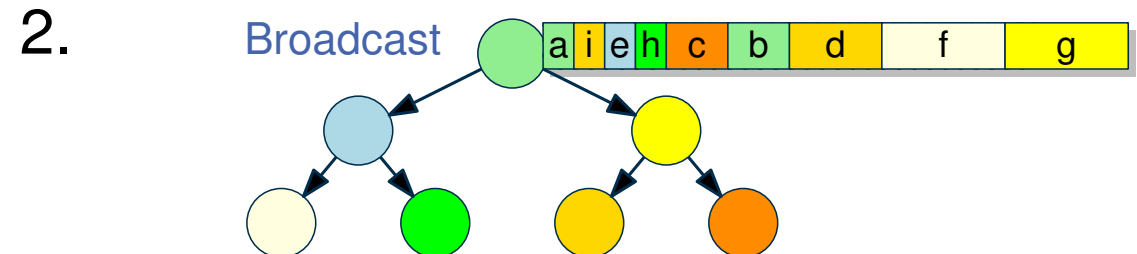
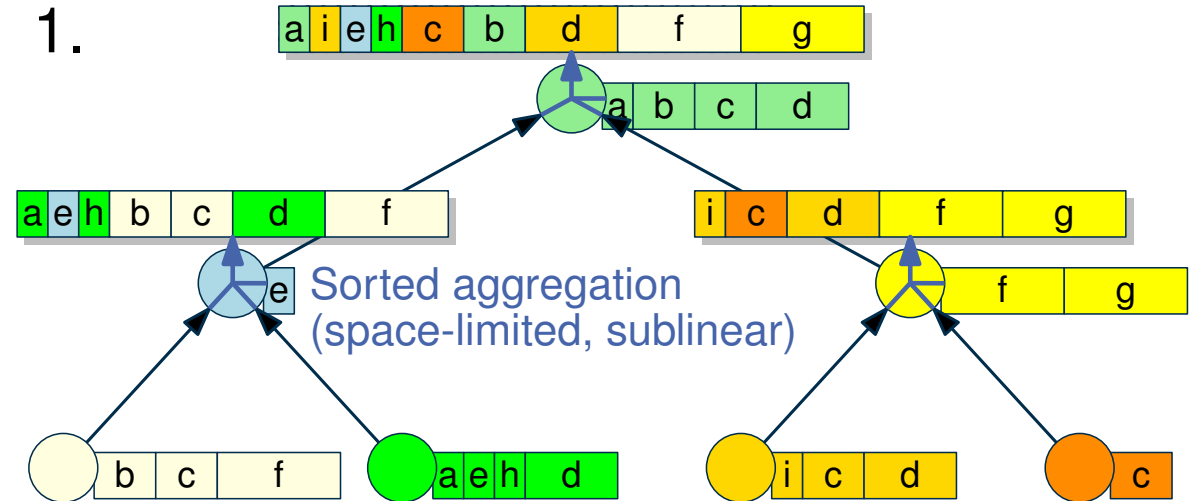
- Aggregate information along binary tree of processors
- Detect duplicates during merge
- Result is of compact shape
- Sublinear buffer size growth:
Discard longest clauses as necessary



Clause Exchange in Mallob

Custom collective operation [SAT'21]

- Aggregate information along binary tree of processors
- Detect duplicates during merge
- Result is of compact shape
- Sublinear buffer size growth:
Discard longest clauses as necessary



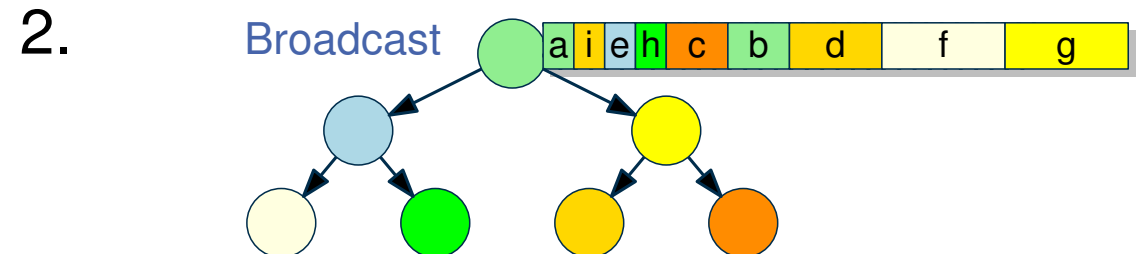
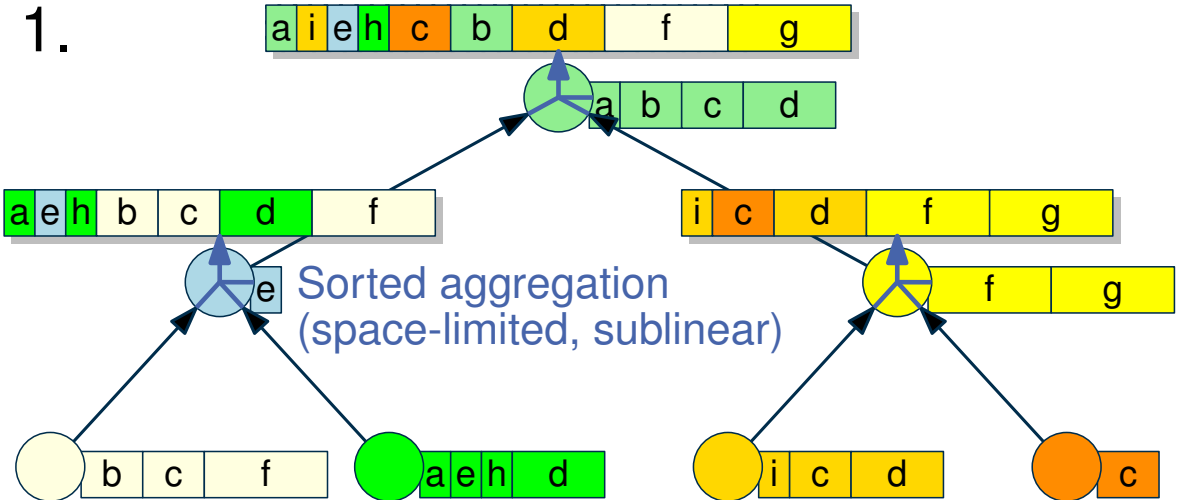
Clause Exchange in Mallob

Custom collective operation [SAT'21]

- Aggregate information along binary tree of processors
- Detect duplicates during merge
- Result is of compact shape
- Sublinear buffer size growth:
Discard longest clauses as necessary

Observations

- Clause needs to meet global quality threshold to be shared successfully
- Quality threshold adapts to state of solving



Clause Filtering

The Problem

Given a shared clause c and a solver S , decide if S has received or produced c before (recently).

Previously: [HordeSat] [SAT'21]

- Bloom filters: fixed size, risk of false positives

Clause Filtering

The Problem

Given a shared clause c and a solver S , decide if S has received or produced c before (recently).

Previously: [HordeSat] [SAT'21]

- Bloom filters: fixed size, risk of false positives

Mallob'22+: Exact distributed filter [ISC'22]

- Process p remembers clauses it exported itself and tags their producing solver(s)

Clause Filtering

The Problem

Given a shared clause c and a solver S , decide if S has received or produced c before (recently).

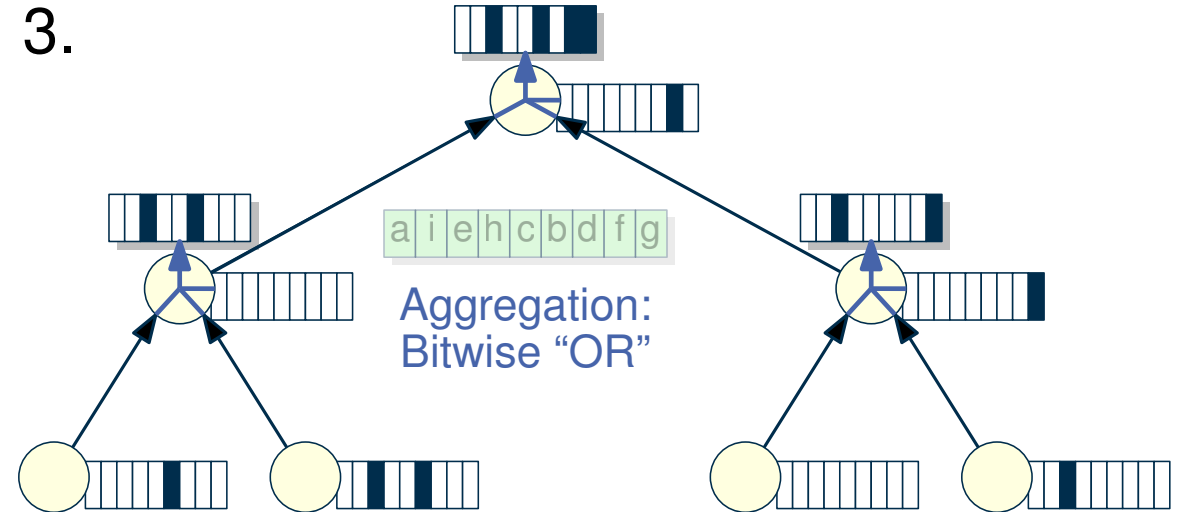
Previously: [HordeSat] [SAT'21]

- Bloom filters: fixed size, risk of **false positives**

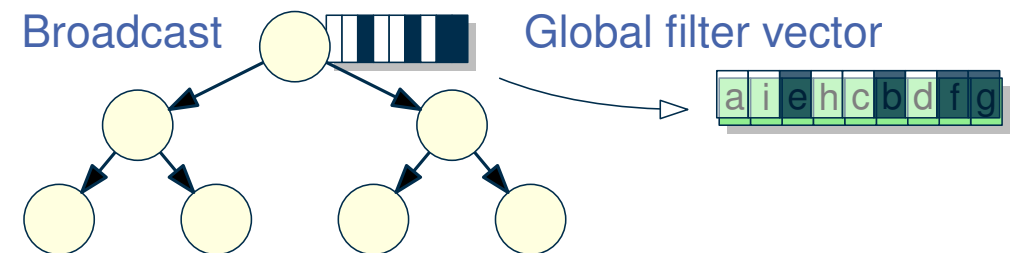
Mallob'22+: Exact distributed filter [ISC'22]

- Process p remembers clauses it exported itself and tags their producing solver(s)
- Aggregate bit vector v where $v[i] := \bigvee_p (p \text{ remembers } c_i)$
- Only import clauses c_i for which $v[i] = \text{false}$

3.



4.



Clause Filtering

The Problem

Given a shared clause c and a solver S , decide if S has received or produced c before (recently).

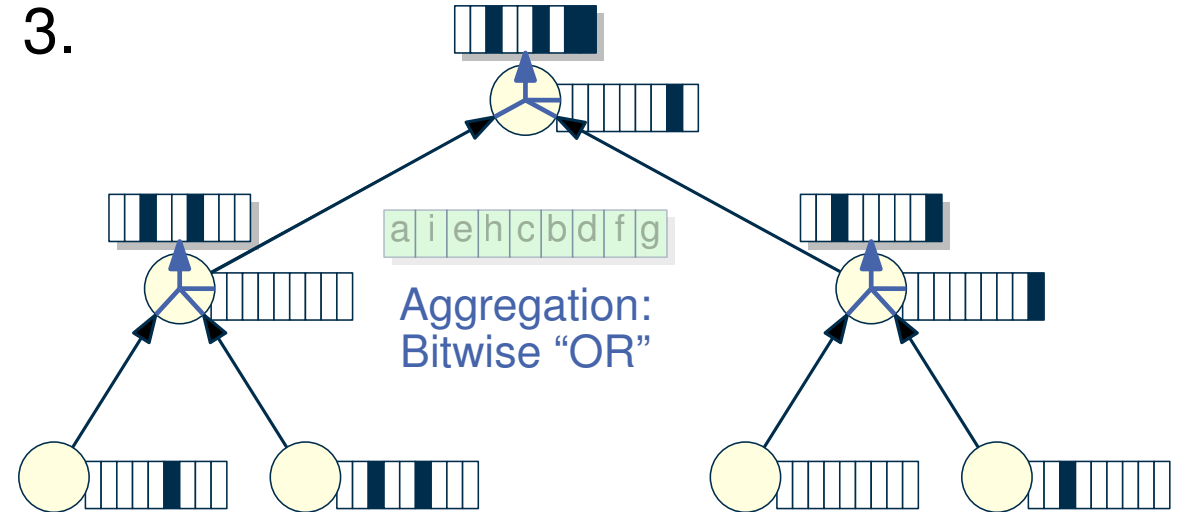
Previously: [HordeSat] [SAT'21]

- Bloom filters: fixed size, risk of **false positives**

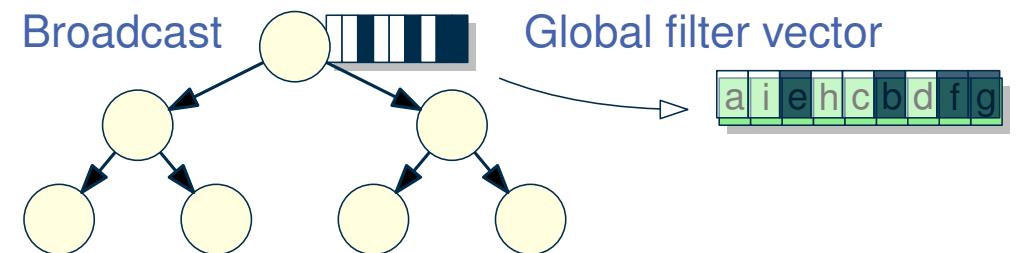
Mallob'22+: Exact distributed filter [ISC'22]

- Process p remembers clauses it exported itself and tags their producing solver(s)
- Aggregate bit vector v where $v[i] := \bigvee_p (p \text{ remembers } c_i)$
- Only import clauses c_i for which $v[i] = \text{false}$
- Use producer data to prevent **mirroring clauses back** to their producer(s)

3.



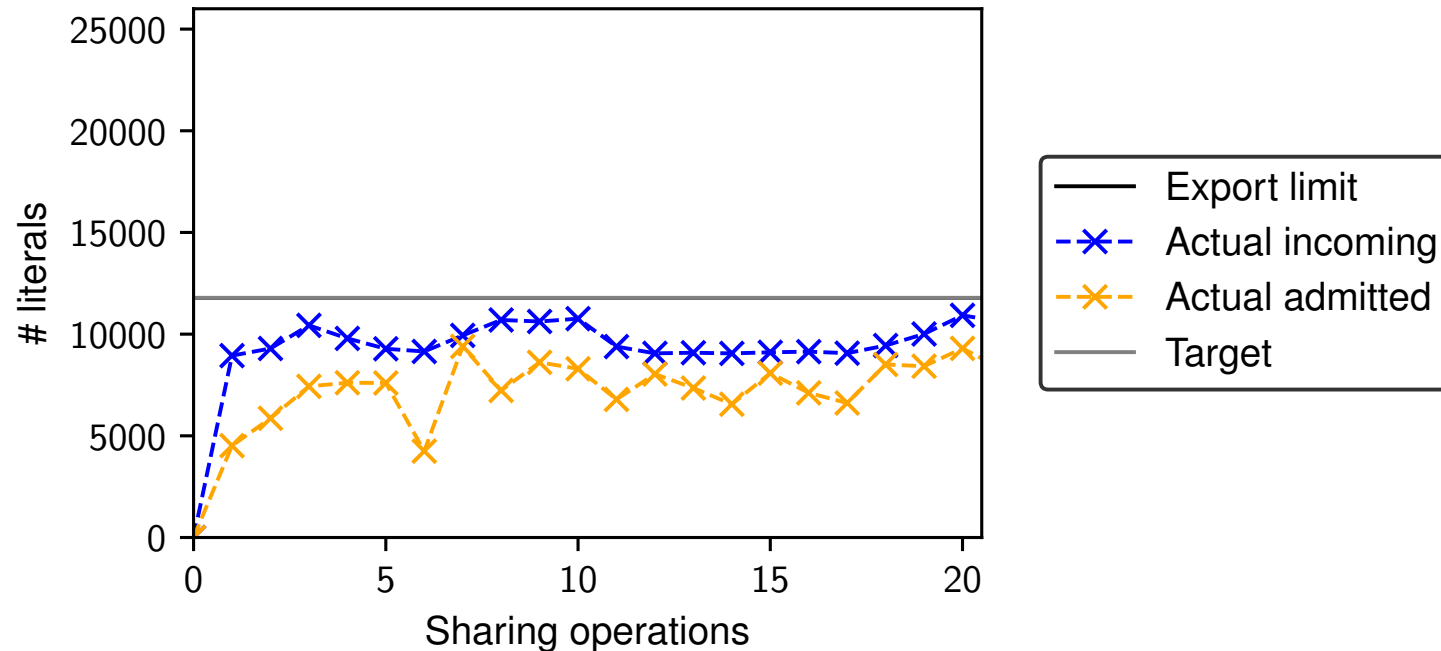
4.



Enforcing a Sharing Volume

We want to share L literals per sharing but may only get $L' < L$ successfully shared literals. Why?

1. Processes didn't produce, export enough clauses
2. Duplicate clauses were detected and eliminated during aggregation
3. Distributed filter blocked some of the transmitted clauses

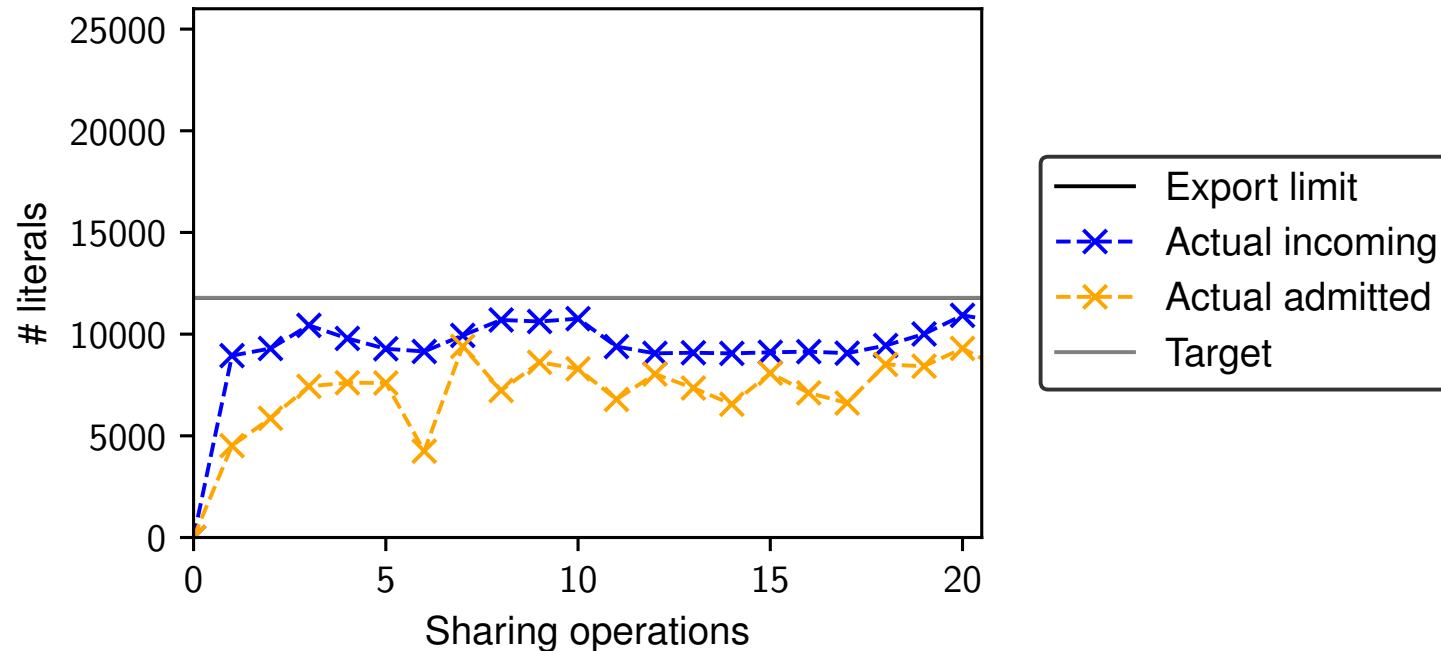


Enforcing a Sharing Volume

We want to share L literals per sharing but may only get $L' < L$ successfully shared literals. Why?

1. Processes didn't produce, export enough clauses
2. Duplicate clauses were detected and eliminated during aggregation
3. Distributed filter blocked some of the transmitted clauses

Fix: Elastic compensation for sharing volume unused for algorithmic reasons (2., 3.)

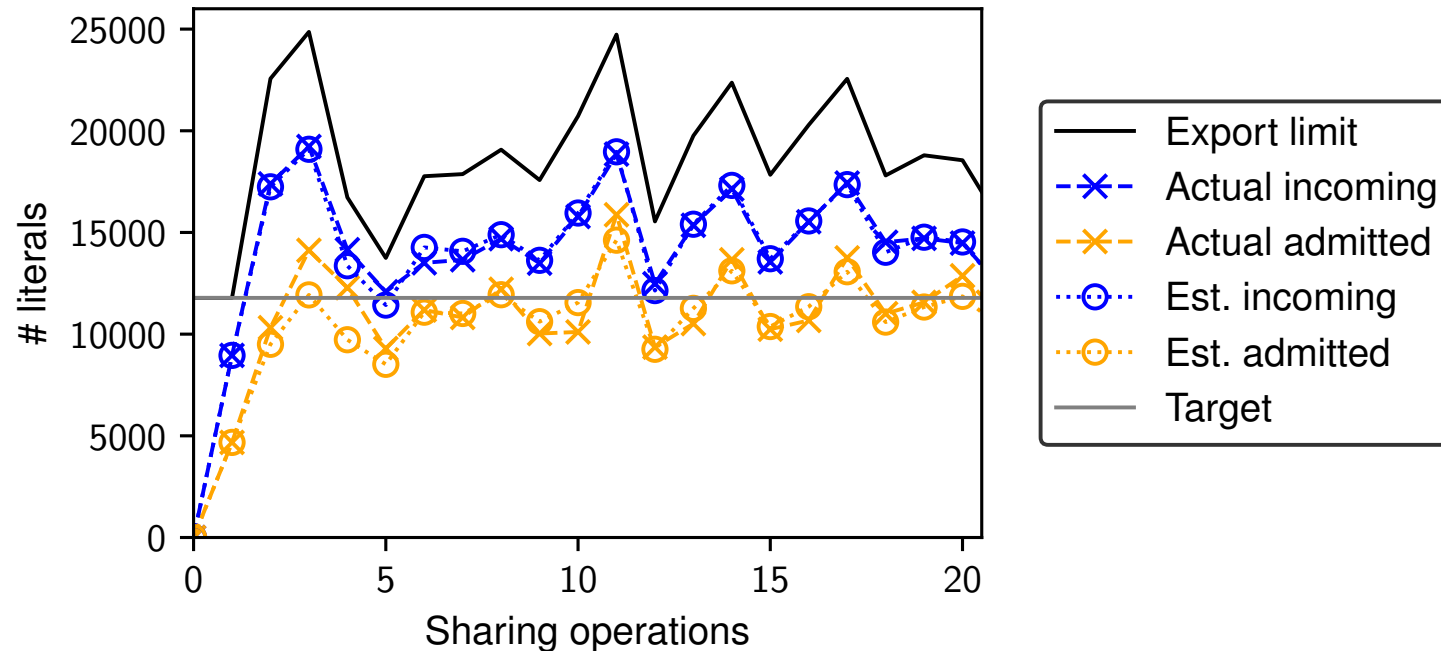


Enforcing a Sharing Volume

We want to share L literals per sharing but may only get $L' < L$ successfully shared literals. Why?

1. Processes didn't produce, export enough clauses
2. Duplicate clauses were detected and eliminated during aggregation
3. Distributed filter blocked some of the transmitted clauses

Fix: Elastic compensation for sharing volume unused for algorithmic reasons (2., 3.)



Handling LBD Values

- Seq. solving: central metric for whether to keep a clause
- **But:** LBD found by solver A not necessarily meaningful for solver B! → not as “global” as clause length

Handling LBD Values

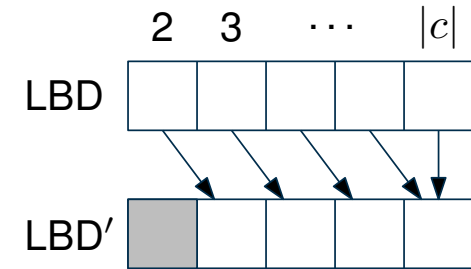
- Seq. solving: central metric for whether to keep a clause
- **But:** LBD found by solver A not necessarily meaningful for solver B! → not as “global” as clause length
- Some solvers keep clauses with LBD 2 indefinitely
 - but expect a single solver’s clause volume!
 - ⇒ Growing overhead (time, space) from low-LBD clauses

Handling LBD Values

- Seq. solving: central metric for whether to keep a clause
- **But:** LBD found by solver A not necessarily meaningful for solver B! → not as “global” as clause length
- Some solvers keep clauses with LBD 2 indefinitely — but expect a single solver’s clause volume!
⇒ Growing overhead (time, space) from low-LBD clauses

Sensible approach: Increment each LBD before import

- Maintains LBD-based prioritization of clauses
- Solver keeps more control over its LBD-2-clauses



	Median RAM	PAR-2
Orig. LBD	108.8 GiB	75.7
Reset LBD	95.6 GiB	74.3
LBD++	97.3 GiB	72.9

768 cores × 349 instances × 300 s

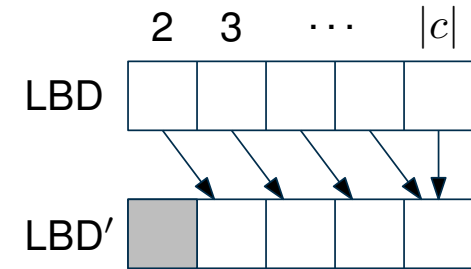
Handling LBD Values

- Seq. solving: central metric for whether to keep a clause
- **But:** LBD found by solver A **not necessarily meaningful** for solver B! → **not as “global”** as clause length
- Some solvers keep clauses with LBD 2 indefinitely — but expect **a single solver’s clause volume!**
⇒ **Growing overhead** (time, space) from low-LBD clauses

Sensible approach: Increment each LBD before import

- Maintains LBD-based prioritization of clauses
- Solver keeps more control over its LBD-2-clauses

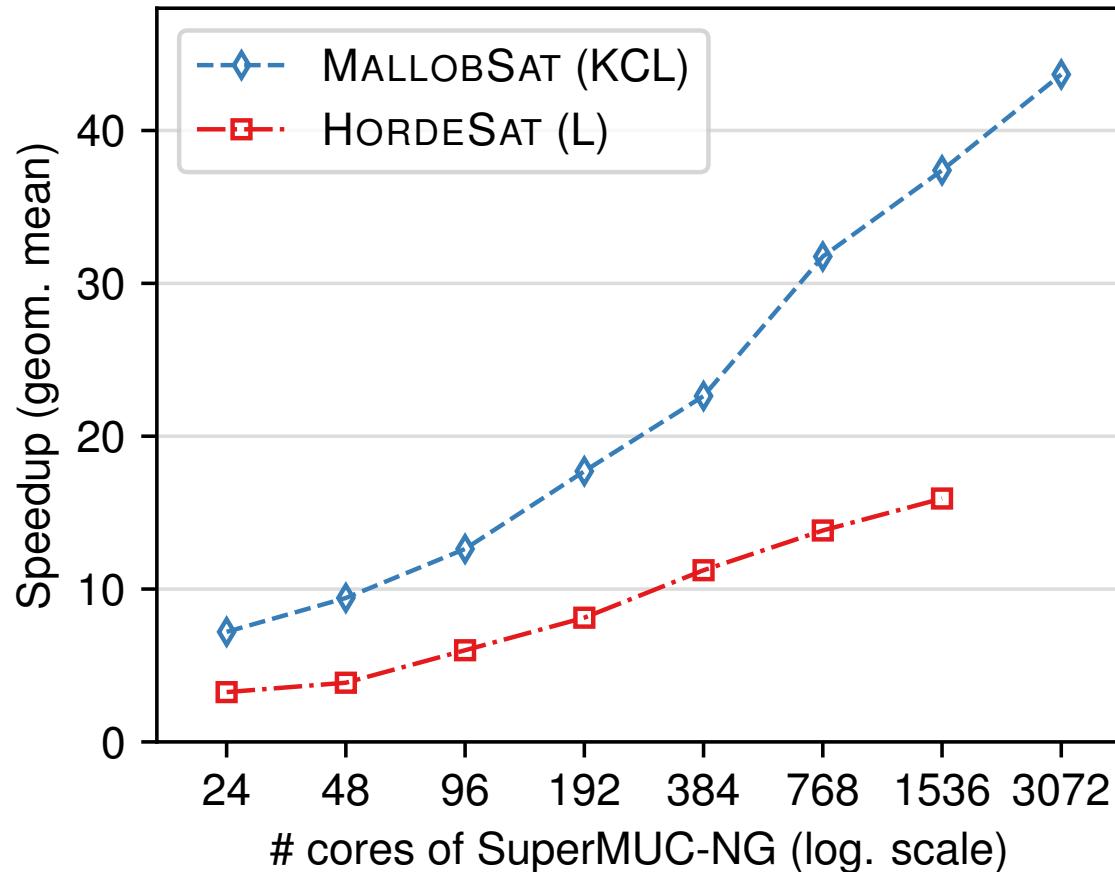
But: Dropping LBD values altogether performs just as well in latest experiments



	Median RAM	PAR-2
Orig. LBD	108.8 GiB	75.7
Reset LBD	95.6 GiB	74.3
LBD++	97.3 GiB	72.9

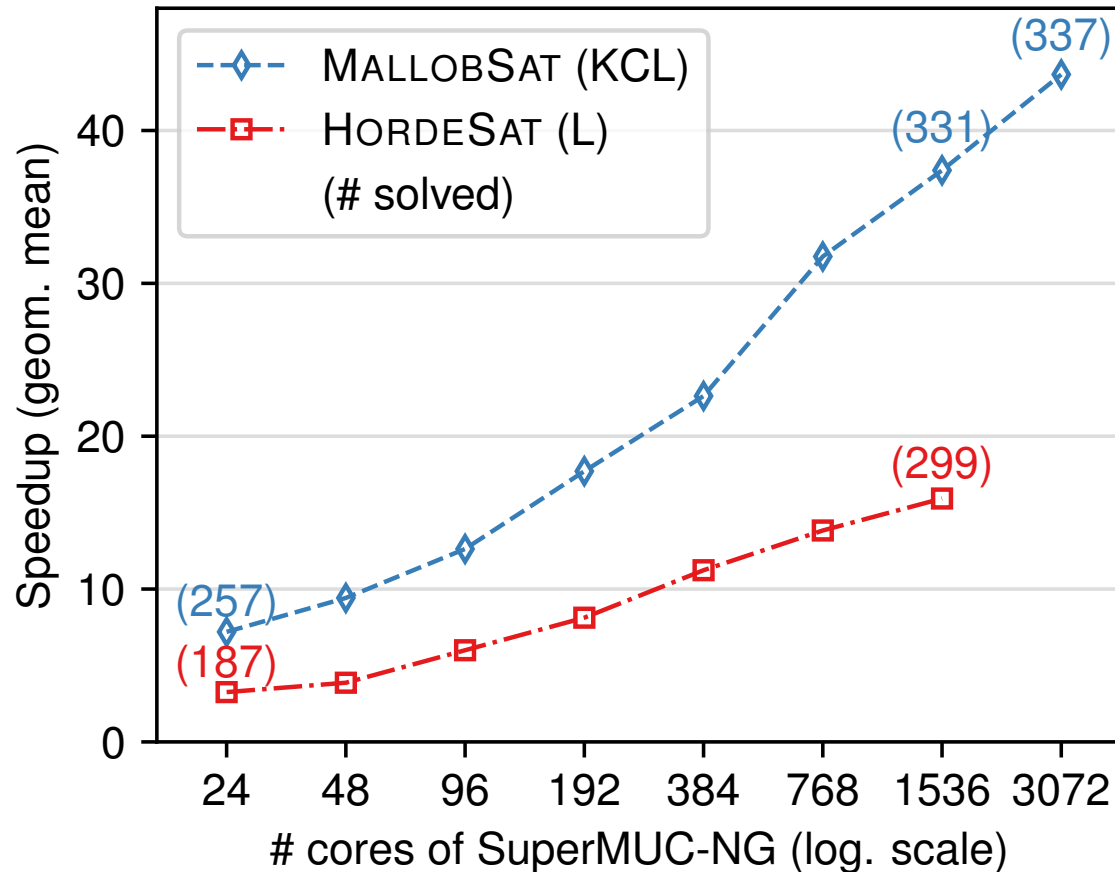
768 cores × 349 instances × 300 s

Scaling of MallobSat



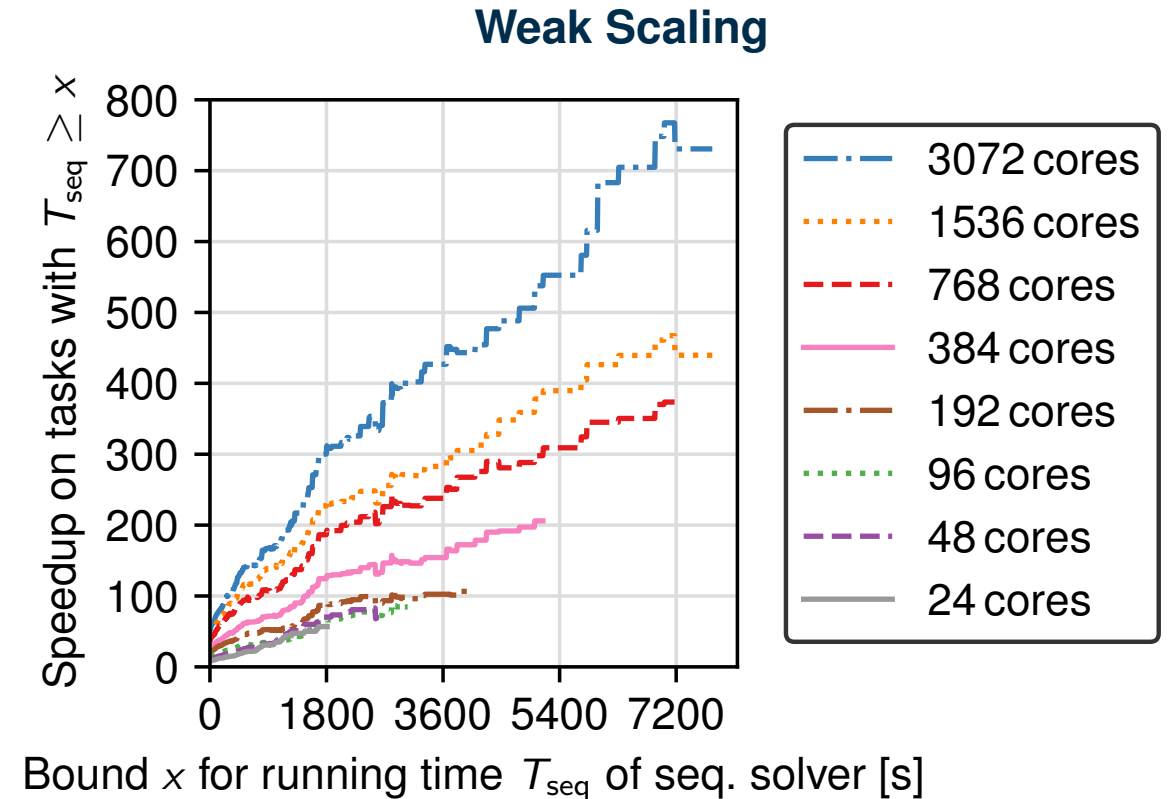
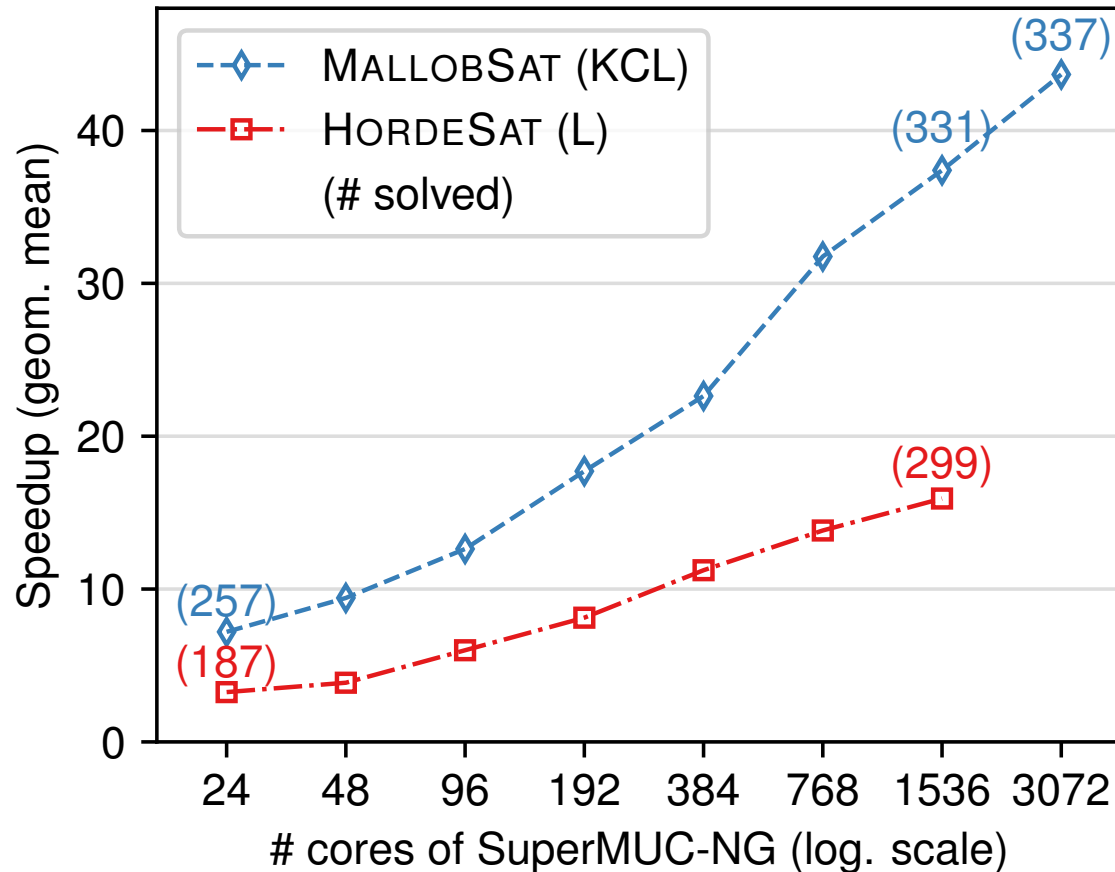
400 problems from SAT Comp. 2021 · Seq. baseline KISSAT_MAB-HYWALK · Seq. limit 32 h (331 solved) · Par. limit 300 s

Scaling of MallobSat



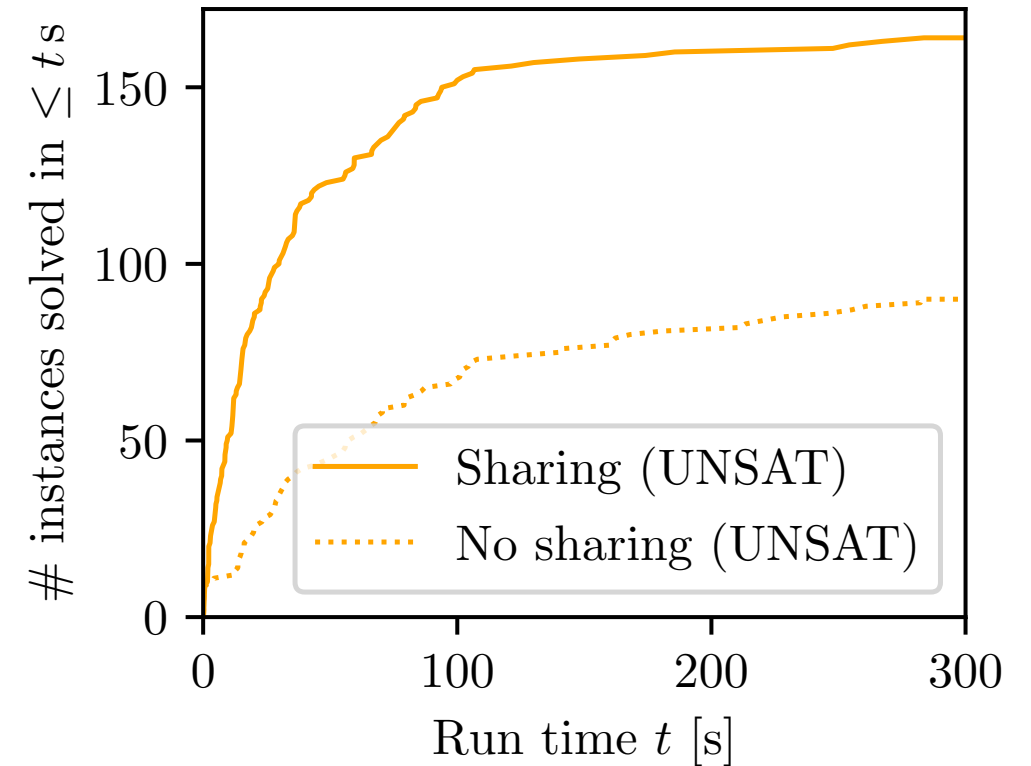
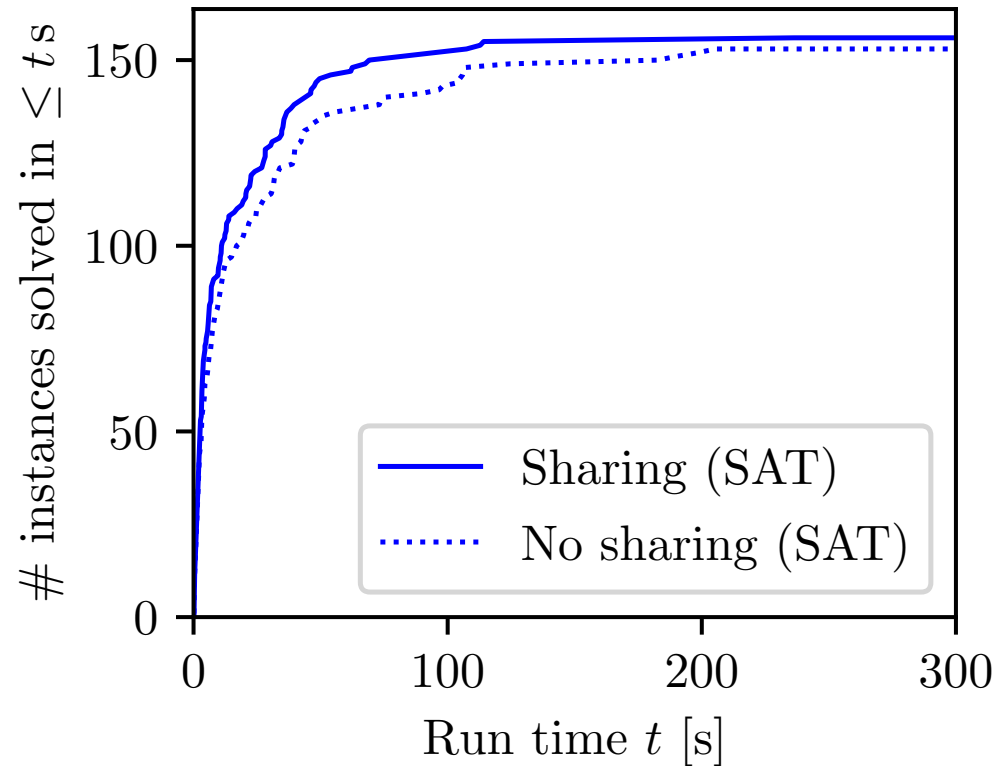
400 problems from SAT Comp. 2021 · Seq. baseline KISSAT_MAB-HYWALK · Seq. limit 32 h (331 solved) · Par. limit 300 s

Scaling of MallobSat



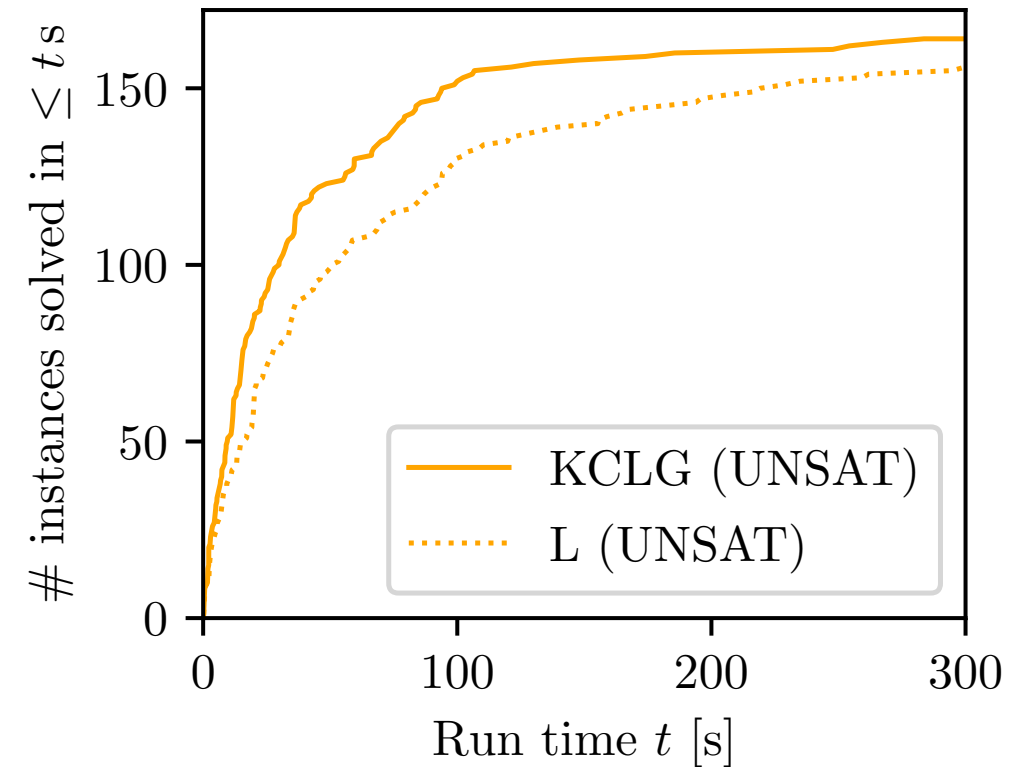
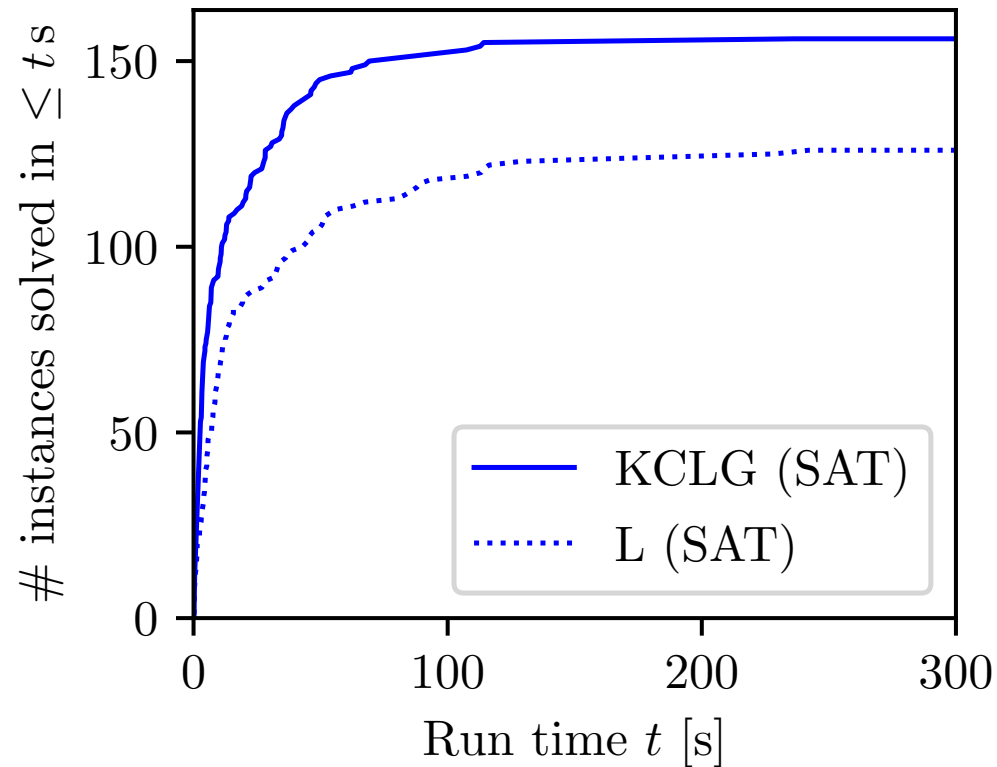
400 problems from SAT Comp. 2021 · Seq. baseline KISSAT_MAB-HYWALK · Seq. limit 32 h (331 solved) · Par. limit 300 s

Merit of Clause Sharing, SAT vs. UNSAT



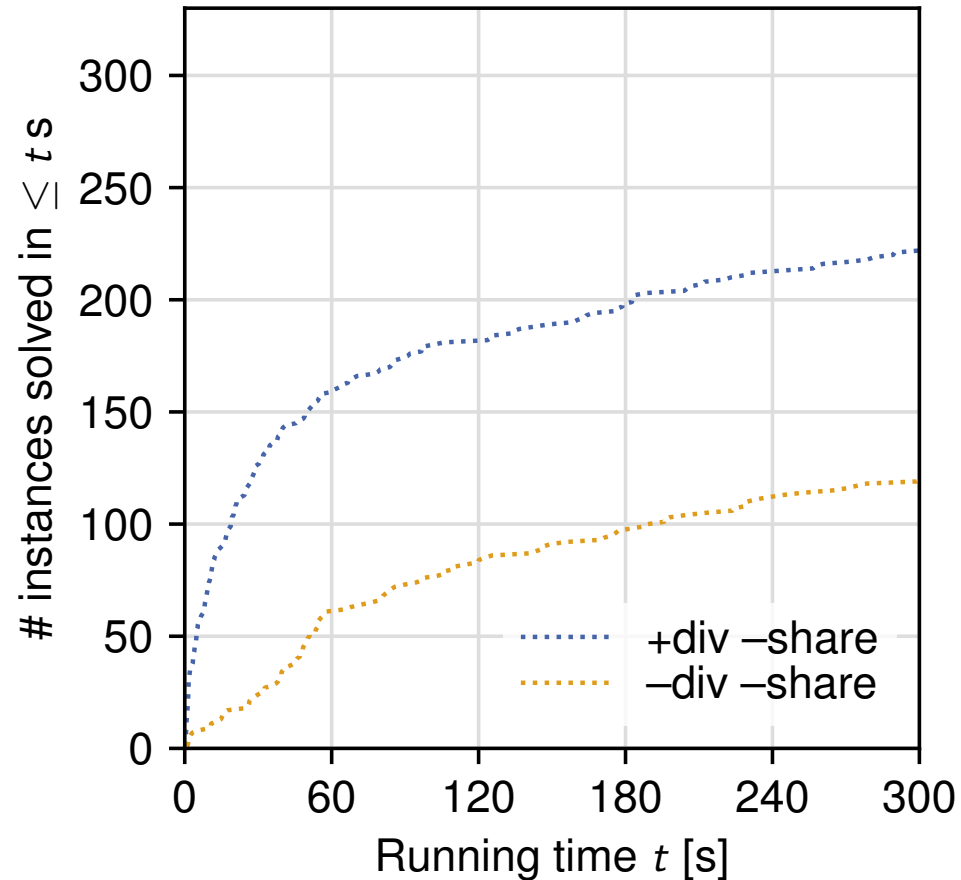
768 cores $\times$ 349 “solvable” instances from ISC 2022 $\times$ 300 s, portfolio “KCLG”

Merit of Diverse Portfolio, SAT vs. UNSAT



768 cores $\times$ 349 “solvable” instances from ISC 2022 $\times$ 300 s, with clause sharing

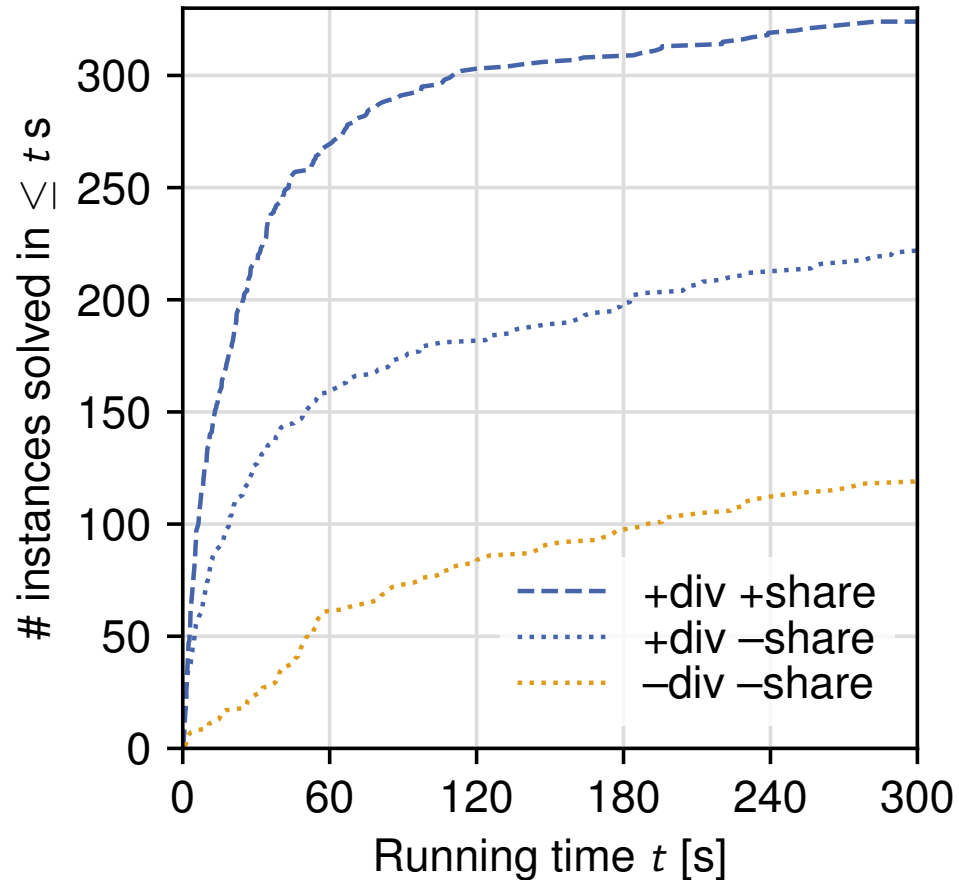
Diversification vs. Sharing @ 768 Cores [JAIR'24]



■ Without clause sharing, diversification is highly effective

349 problems from SAT Comp. 2022 · CaDiCaL only

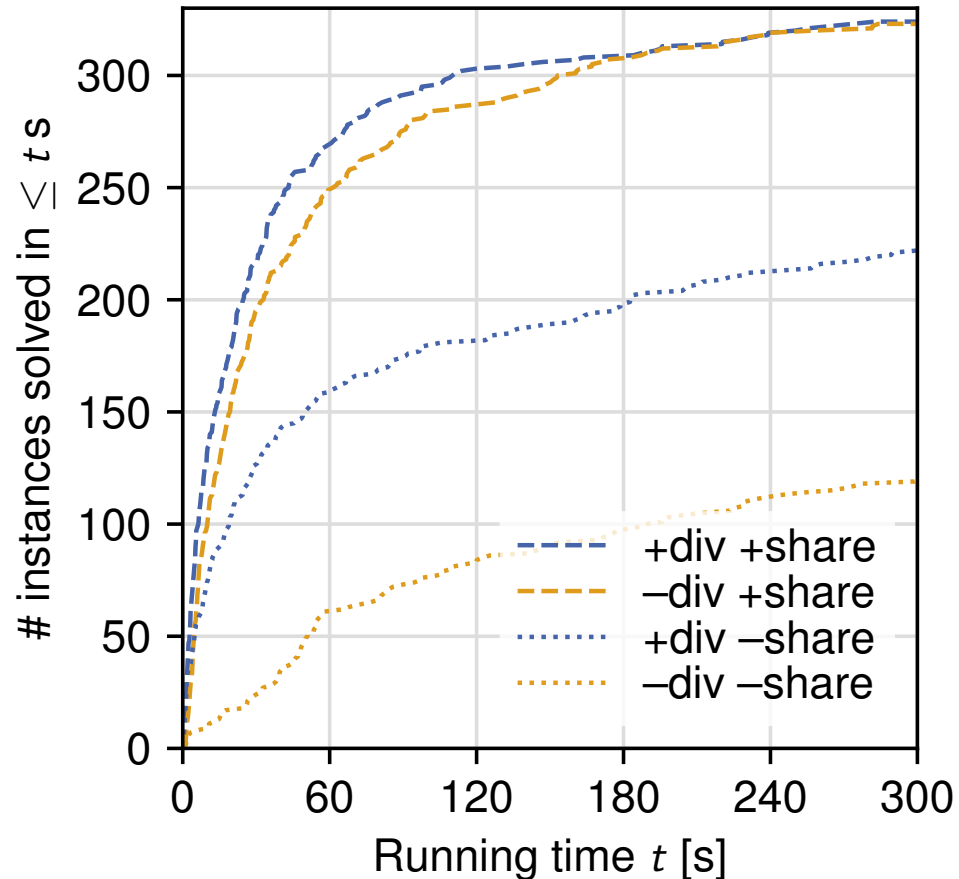
Diversification vs. Sharing @ 768 Cores [JAIR'24]



- Without clause sharing, diversification is highly effective
- With sharing:

349 problems from SAT Comp. 2022 · CaDiCaL only

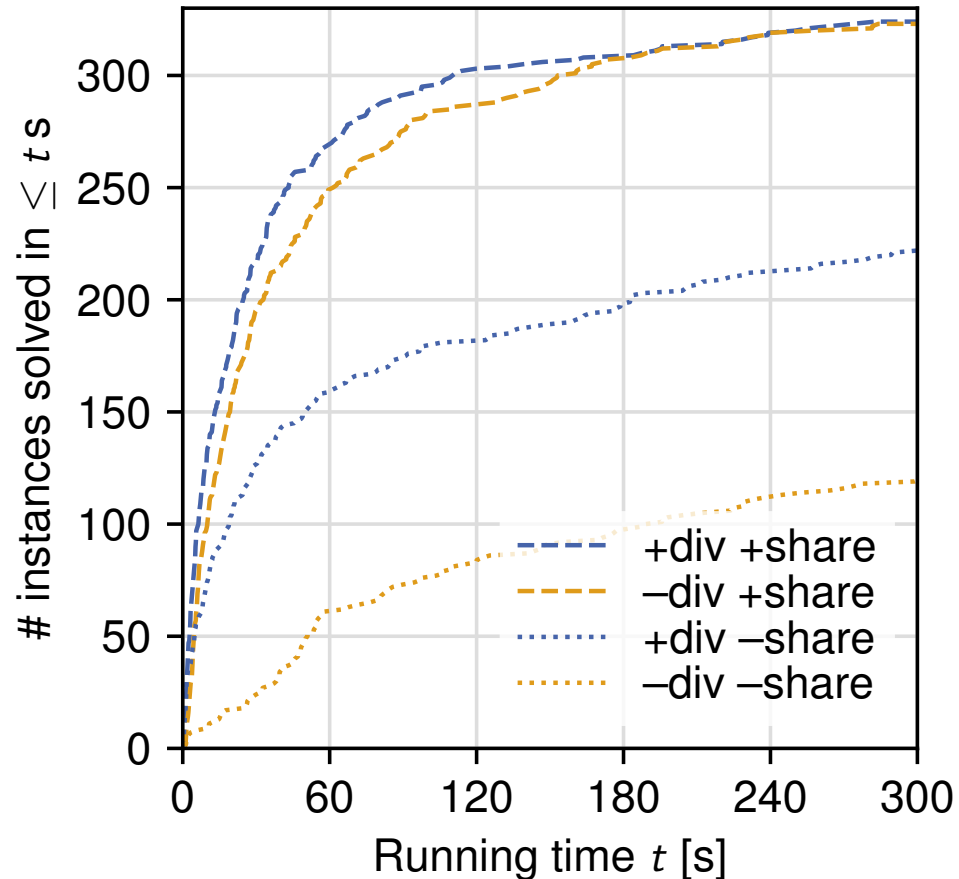
Diversification vs. Sharing @ 768 Cores [JAIR'24]



- Without clause sharing, diversification is highly effective
- With sharing: 768× the same program performs well?!

349 problems from SAT Comp. 2022 · CaDiCaL only

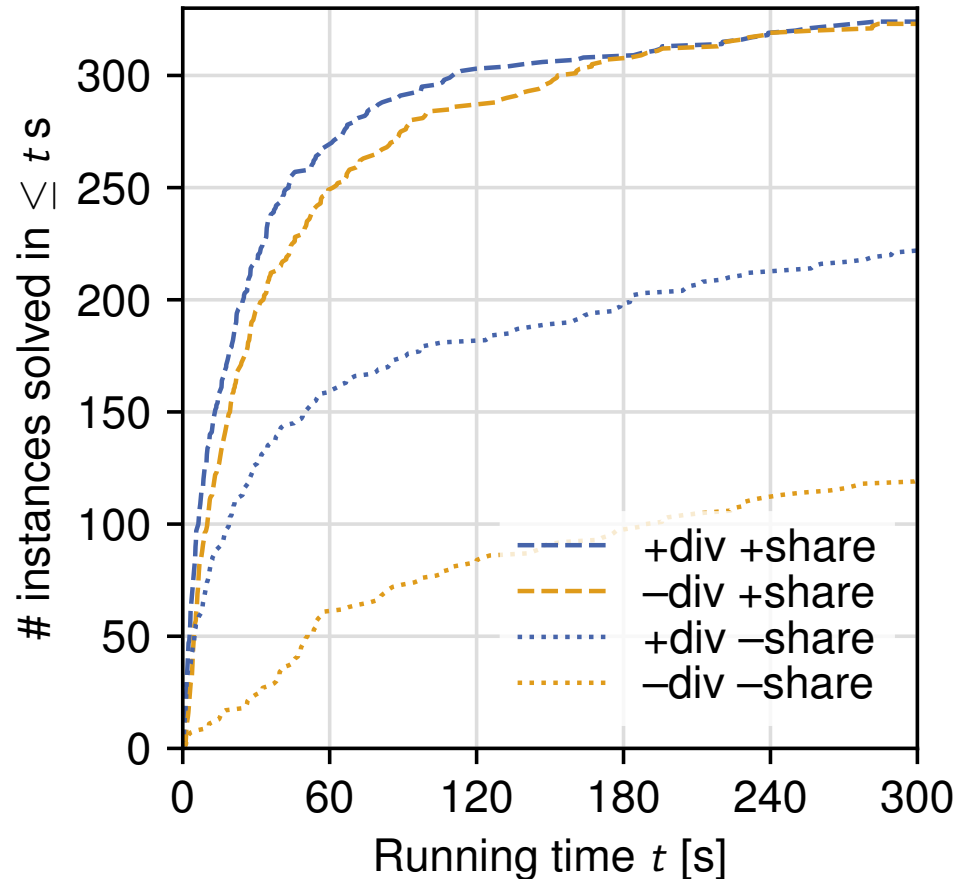
Diversification vs. Sharing @ 768 Cores [JAIR'24]



- Without clause sharing, diversification is highly effective
- With sharing: $768\times$ the same program performs well?!
 - ⇒ Clause imports deviate due to parallel execution
 - ⇒ “Butterfly effect” → effective sharing!

349 problems from SAT Comp. 2022 · CaDiCaL only

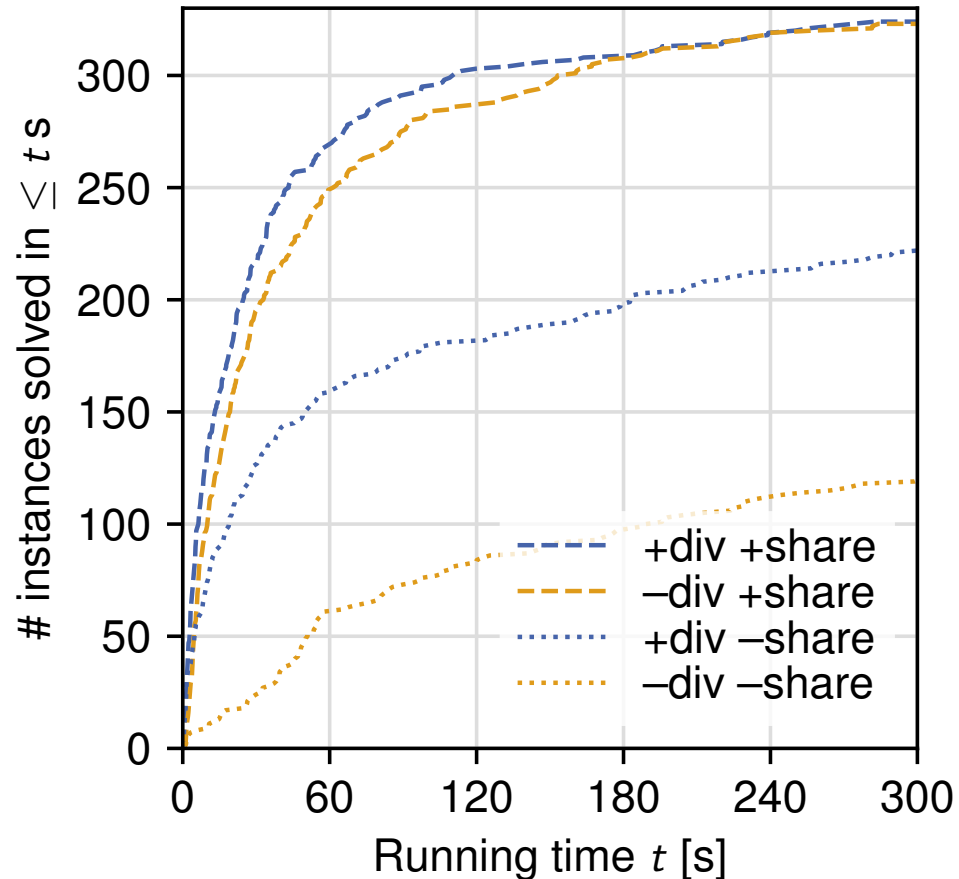
Diversification vs. Sharing @ 768 Cores [JAIR'24]



- Without clause sharing, diversification is highly effective
- With sharing: $768\times$ the same program performs well?!
 - ⇒ Clause imports deviate due to parallel execution
 - ⇒ “Butterfly effect” → effective sharing!
- Similar findings @ 3072 cores
 - Default CADICAL with primitive diversification (seeds, phases) performs competitively
 - Fully diversified portfolio without clause sharing does not

349 problems from SAT Comp. 2022 · CaDiCaL only

Diversification vs. Sharing @ 768 Cores [JAIR'24]



- Without clause sharing, diversification is highly effective
 - With sharing: 768× the same program performs well?!
 - ⇒ Clause imports deviate due to parallel execution
 - ⇒ “Butterfly effect” → effective sharing!
 - Similar findings @ 3072 cores
 - Default CADICAL with primitive diversification (seeds, phases) performs competitively
 - Fully diversified portfolio without clause sharing does not
- ⇒ Portfolio of diverse configurations is dispensable
- ⇒ Clause sharing is essential

349 problems from SAT Comp. 2022 · CaDiCaL only

MallobSat: A Portfolio Solver?

Prevalent concept in literature: **Portfolio solver** with clause sharing / **Clause-sharing portfolio**

- “a problem instance is independently given to a collection of solvers *competing* for a solution in parallel”
– Fichte et al., 2023
- “each thread runs a *different SAT solver* on the same instance[, which] *in combination with* clause-sharing leads to surprisingly good performance *for small portfolio sizes*”
– Ozdemir et al., 2021

MallobSat: A Portfolio Solver?

Prevalent concept in literature: **Portfolio solver** with clause sharing / **Clause-sharing portfolio**

- “a problem instance is independently given to a collection of solvers **competing** for a solution in parallel”
– Fichte et al., 2023
- “each thread runs a **different SAT solver** on the same instance[, which] **in combination with clause-sharing** leads to **surprisingly good performance for small portfolio sizes**”
– Ozdemir et al., 2021

Our view, based on empirical observations:

MALLOBSAT is a Clause-sharing solver with diversification

- Clause sharing = main driver of scalability
- Adding explicit diversification is **beneficial** but not essential
- Applicability to other solvers?

Better Efficiency?

Massive parallelism for a single formula

- Faster solving times
- Can resolve problems out of reach for sequential solvers
- Not that resource efficient (on average)

Better Efficiency?

Massive parallelism for a single formula

- Faster solving times
- Can resolve problems out of reach for sequential solvers
- Not that resource efficient (on average)

Solving many formulas in parallel

- Embarrassingly parallel
- Solving itself less powerful

Better Efficiency?

Massive parallelism for a single formula

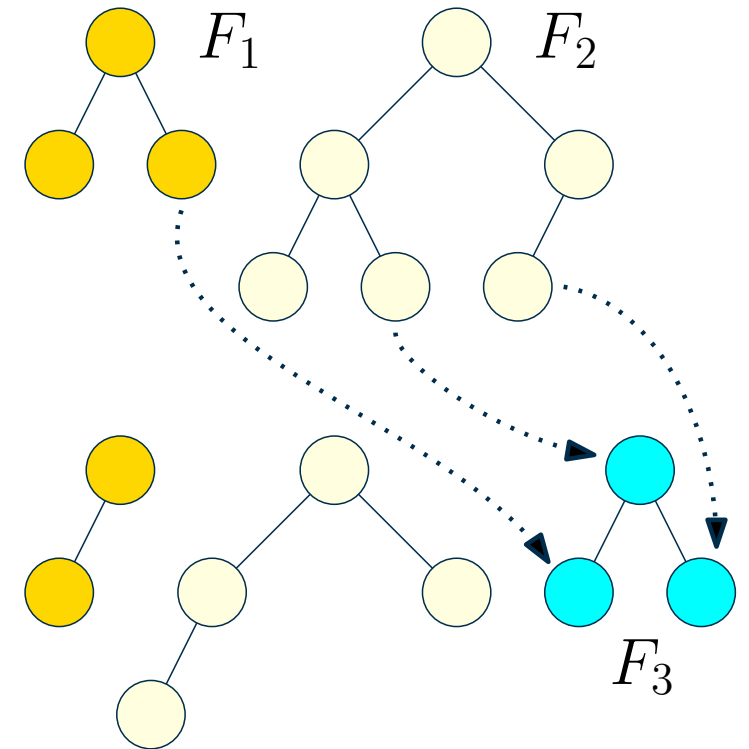
- Faster solving times
- Can resolve problems out of reach for sequential solvers
- Not that resource efficient (on average)

Solving many formulas in parallel

- Embarrassingly parallel
- Solving itself less powerful

Best of both worlds? [EuroPar'22]

- On demand scheduling of incoming (SAT) jobs
- Resize jobs during their execution as needed
- Few milliseconds to schedule an incoming job, full utilization whenever sufficient demand is present



Scheduling Many SAT Tasks [JAIR'24]

Problem statement

Given $x \in \{400, 1600, 6400\}$ cores and 400 SAT tasks, solve as many tasks as possible.

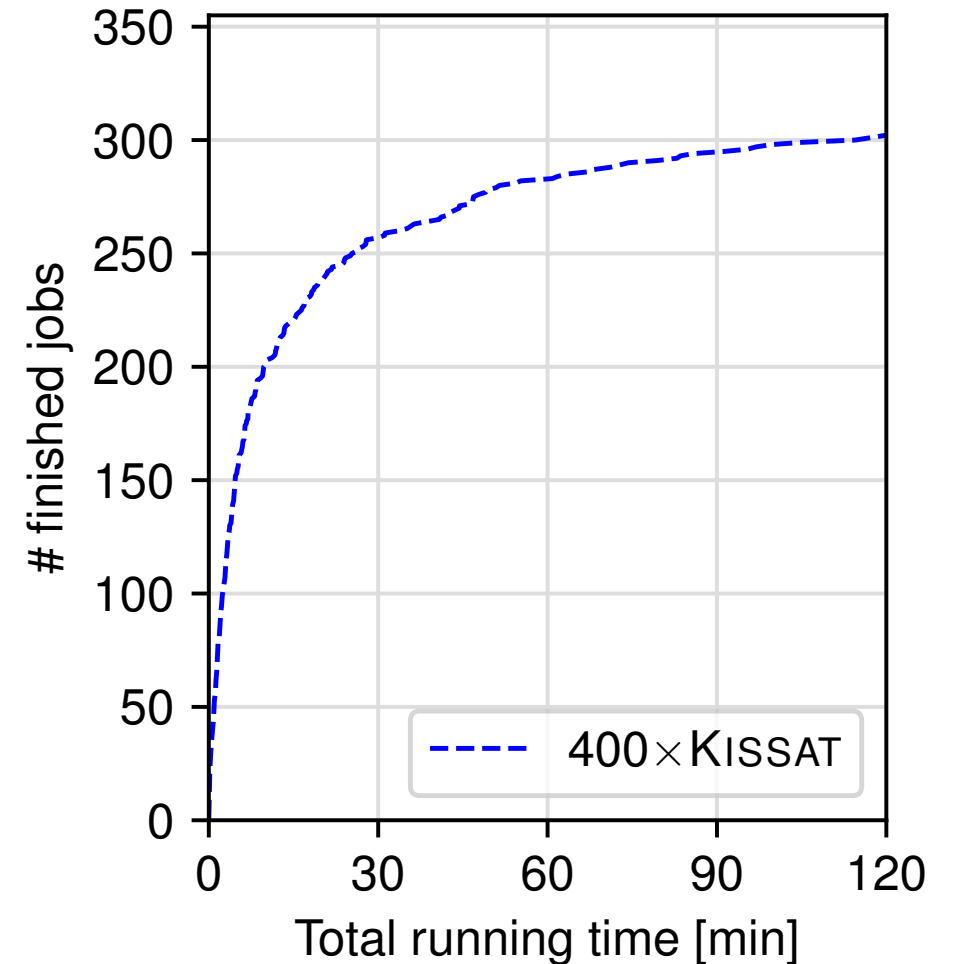
Scheduling Many SAT Tasks [JAIR'24]

Problem statement

Given $x \in \{400, 1600, 6400\}$ cores and 400 SAT tasks, solve as many tasks as possible.

Extreme 1: 400 sequential solvers

- “Embarrassingly parallel” job processing
- Highly efficient (no redundant work)
- Sequential solving only → no scaling opportunity



Scheduling Many SAT Tasks [JAIR'24]

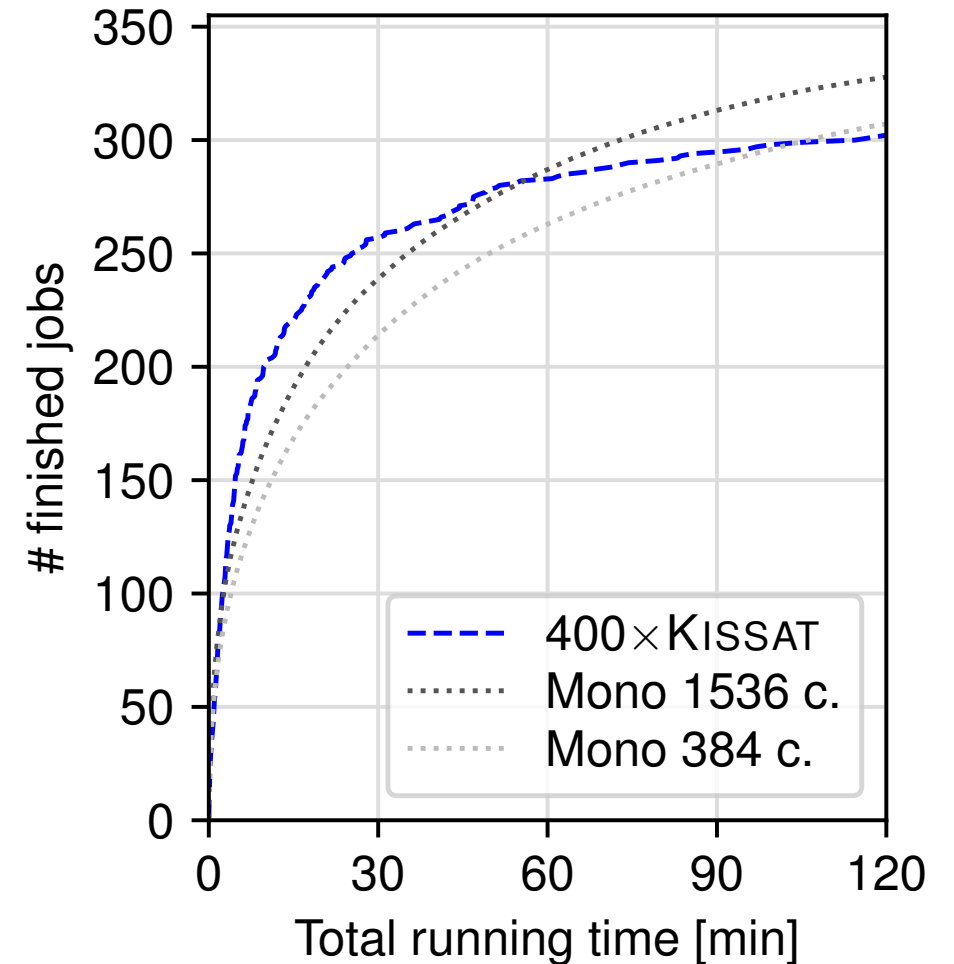
Problem statement

Given $x \in \{400, 1600, 6400\}$ cores and 400 SAT tasks, solve as many tasks as possible.

Extreme 1: 400 sequential solvers ($400 \times$ Kissat)

Extreme 2: “Monolithic” parallel solving

- One job at a time
- Generous assumption: instances sorted by run time
- Maximizes per-instance speedups
- No task parallelism, poor efficiency



Scheduling Many SAT Tasks [JAIR'24]

Problem statement

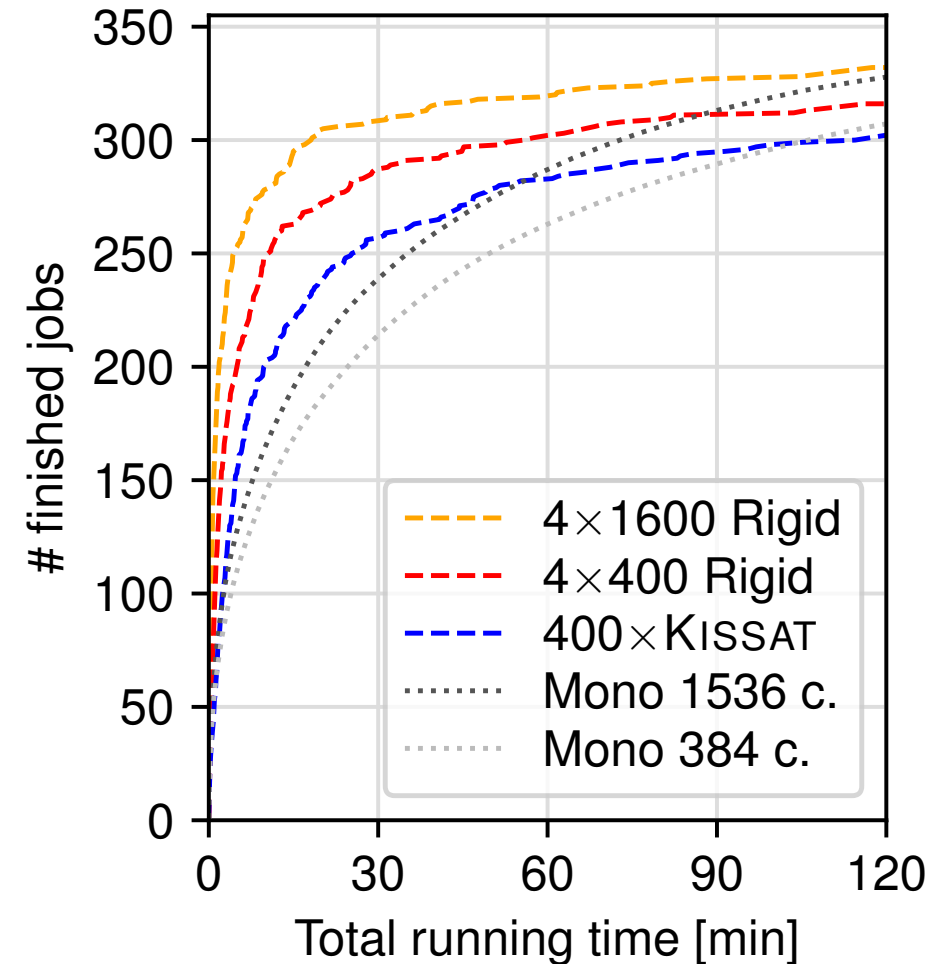
Given $x \in \{400, 1600, 6400\}$ cores and 400 SAT tasks, solve as many tasks as possible.

Extreme 1: 400 sequential solvers ($400 \times$ Kissat)

Extreme 2: “Monolithic” parallel solving

Middle ground 1: Divide cores evenly among jobs

- Solid speedups at small-scale parallel SAT
- After 15 min, > 50% of cores are idling



Scheduling Many SAT Tasks [JAIR'24]

Problem statement

Given $x \in \{400, 1600, 6400\}$ cores and 400 SAT tasks, solve as many tasks as possible.

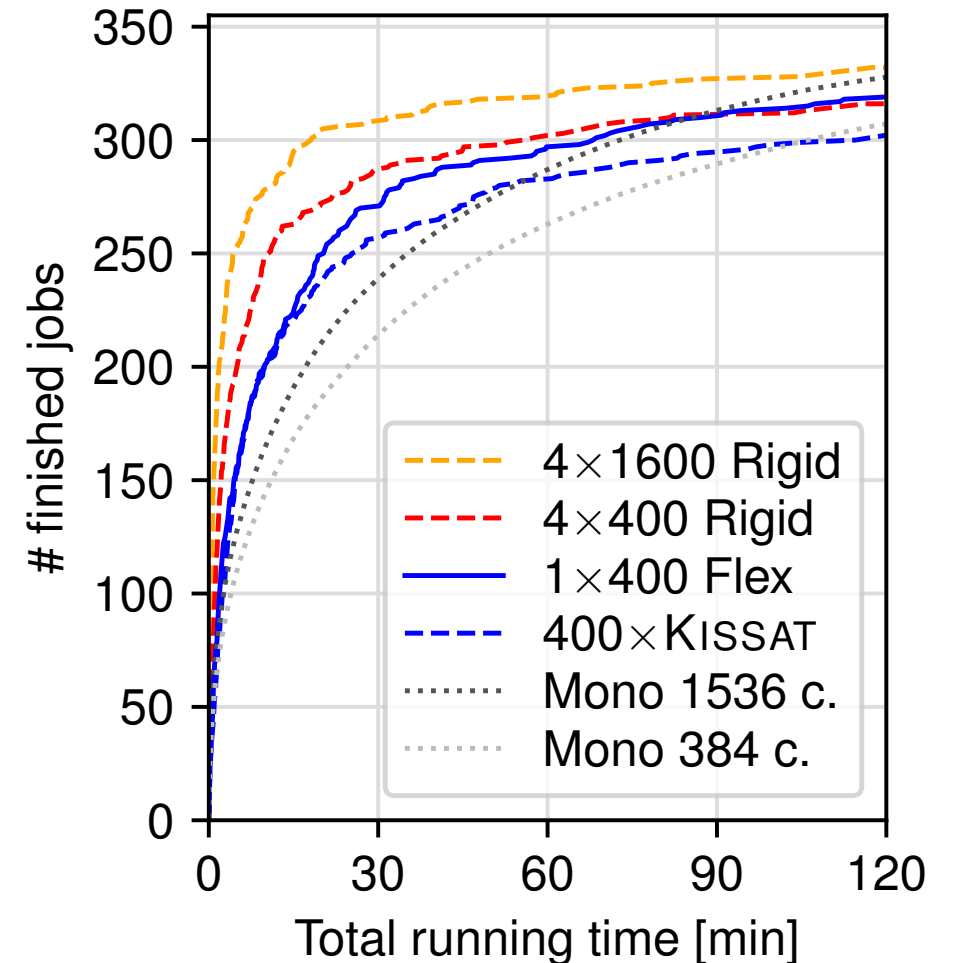
Extreme 1: 400 sequential solvers ($400 \times$ Kissat)

Extreme 2: “Monolithic” parallel solving

Middle ground 1: Divide cores among jobs (rigid)

Middle ground 2: Divide cores **dynamically** among jobs

- Finishing jobs **yield resources** to remaining jobs



Scheduling Many SAT Tasks [JAIR'24]

Problem statement

Given $x \in \{400, 1600, 6400\}$ cores and 400 SAT tasks, solve as many tasks as possible.

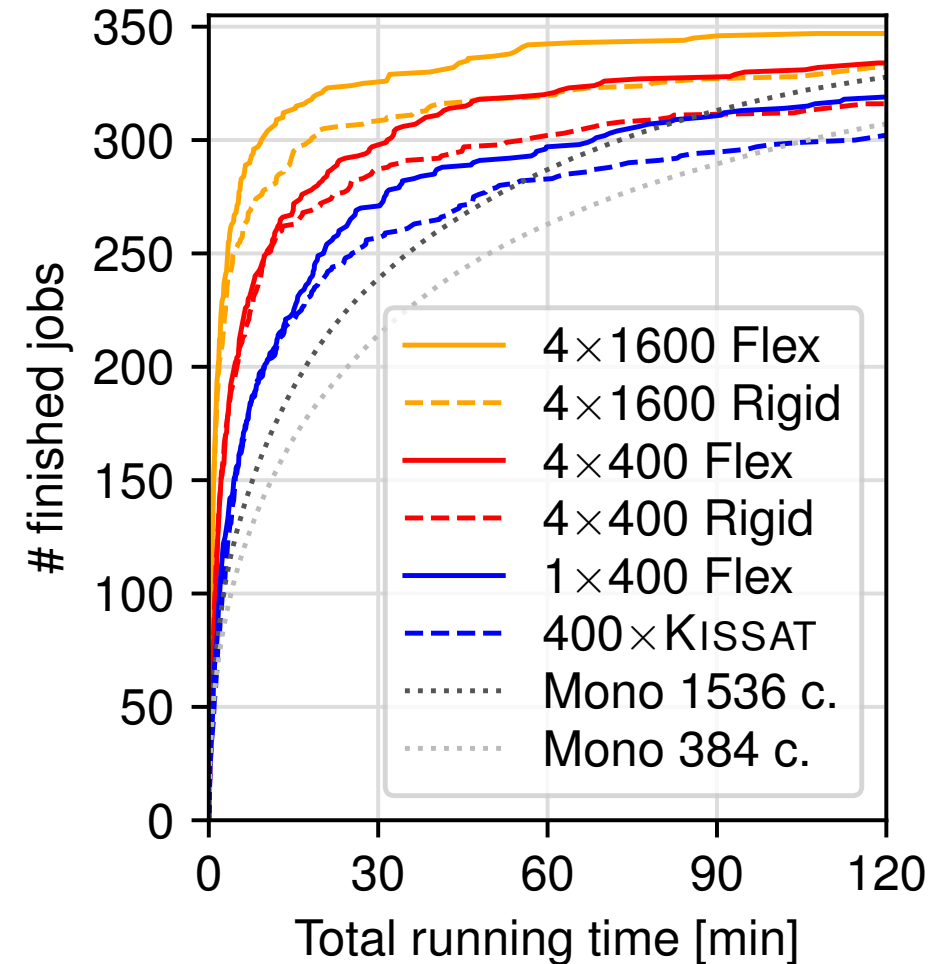
Extreme 1: 400 sequential solvers ($400 \times$ Kissat)

Extreme 2: “Monolithic” parallel solving

Middle ground 1: Divide cores among jobs (rigid)

Middle ground 2: Divide cores *dynamically* among jobs

- Finishing jobs yield resources to remaining jobs
- Record number of solved instances using relatively little resources



Current Frontiers

- Streamlined clause-sharing solver design, away from portfolios
- Address high main memory requirements, especially for huge formulas
- Preprocessing and Inprocessing in parallel SAT solving
- Applying technology to related tools, applications
 - Verification (Bounded Model Checking, SMT Solving)
 - Optimization (MaxSAT Solving)
 - Crucial piece of technology: Distributed incremental SAT solving!
- Uses of GPUs ?? (Stochastic Local Search, Inprocessing offloading, . . .)

Recap

Parallel SAT Solving

- Popular parallelization approaches for SAT (“antisocial nerds” analogy)
 - Search space splitting, Cube&Conquer
 - Pure portfolio
 - Clause sharing portfolio
- All-to-all clause sharing can be very useful and scalable (up and down) if implemented well
 - huge for unsatisfiable, nice-to-have for satisfiable problems
 - diversifies solvers effectively in and of itself
- Exploit embarrassingly parallel job processing for interactive solving & best efficiency

Next Up

- Proof pragmatics: Formats for different proof systems in the wild
- Bringing proof technology to parallel and distributed SAT solving